ENGINEERING BOOTCAMP - WEEK 5 CASE STUDY

Deploying InfinityCastle on AWS

S3, EC2, RDS, Lambda, CloudWatch, Docker, ECR and ECS Fargate

The class notes explained and connected, AWS theory in depth, every design decision, every change to make in the project, and an exact step-by-step runbook with commands and code

What is inside	What was taught in class and how it all connects; AWS from the ground up (global infrastructure, IAM, EC2, VPC, S3, RDS, Lambda, CloudWatch, Docker, ECR, ECS, load balancing, service discovery); design decisions with alternatives; the exact project changes (file, place, code); Milestone 1, 2 and 3 runbooks with checkpoints; security, backups, cost and clean-up; troubleshooting; interview questions; a copy-paste appendix.
Application	InfinityCastle - Issue Tracker: eureka-server, api-gateway, user-service, project-service, issue-service (Spring Boot 4.1, Java 21, Spring Cloud) + React 19 frontend + MySQL.
Decisions taken	One EC2 per application; Lambda = profile-photo processor (S3 upload -> RDS); JARs built in Spring Tools, Docker images built on an EC2 "builder" as in class; region us-east-1.
Important	The project changes (Part 5) are already applied; you build, deploy and commit. Never put passwords, .pem keys or access keys in Git.
Prepared	17 October 2026, for Nishant (Bruno)

Contents

How to use this guide, and the big picture 5

1.1 The case study in one page 5
1.2 What you will build - Milestone 1 5
1.3 What you will build - Milestone 2 6
1.4 Names used in this guide 7
1.5 The order of work 7

The class notes - what was taught and how it all connects 8

2.1 The map: class demo -> case-study step 8
2.2 Cloud fundamentals (notes page 1) 8
2.3 Global infrastructure (page 2) 9
2.4 Virtualization and hypervisors (pages 2-4) 9
2.5 Connecting and hosting (pages 4-11) 9
2.6 Networking (pages 12-18) 10
2.7 Docker, ECR and ECS (pages 19-30) 10
2.8 RDS and S3 (pages 30-32) 11

AWS in depth - every service this deployment uses 12

3.1 The shared responsibility model 12
3.2 IAM - who may do what 12
3.3 EC2 in depth 13
3.4 VPC in depth 13
3.5 Security groups vs network ACLs 14
3.6 S3 in depth 14
3.7 RDS in depth 14
3.8 AWS Lambda in depth 15
3.9 CloudWatch in depth 15
3.10 Docker in depth 16
3.11 ECR and ECS / Fargate in depth 16
3.12 Load balancing and service discovery 16
3.13 The Well-Architected Framework (for the interview) 17

Design decisions, security plan and environments 18

4.1 Decisions and their reasons 18
4.2 How a request travels 19
4.3 Least-privilege IAM 19
4.4 Security-group matrix 19

4.5 Secrets . . . 20
4.6 Environments: dev, test, prod (case-study guideline) . . . 20
4.7 Backups . . . 20
4.8 Authentication (non-functional requirement) . . . 20

The changes in the project . . . 21

5.1 Overview . . . 21
5.2 The aws Spring profile for the three services . . . 22
5.3 The gateway: CORS from configuration . . . 22
5.4 Optional db.properties . . . 23
5.5 The React build for AWS . . . 23
5.6 Dockerfiles (Milestone 2) . . . 23
5.7 The aws/ folder - deployment scripts . . . 24
5.8 .gitignore additions . . . 27
5.9 Build the JARs and check nothing broke locally . . . 27

Milestone 1 - manual deployment, step by step . . . 28

6.0 What is different from the first plan . . . 28
6.1 Your notepad worksheet - what to save, what not to save . . . 29
6.2 Prepare the laptop (once) . . . 31
6.3 Key pair . . . 31
6.4 Network: the VPC (wizard) . . . 31
6.5 IAM role for the EC2 servers . . . 32
6.6 Security groups (final, working rules) . . . 32
6.7 RDS MySQL . . . 32
6.8 The five EC2 application servers . . . 34
6.9 Create the three schemas and the three database users . . . 35
6.10 Deploy user, project, issue and gateway . . . 37
6.11 Verify the backend . . . 38
6.12 The React frontend on S3 . . . 38
6.13 Milestone 1 status and evidence . . . 40
6.14 Lambda: the profile-photo processor (do after Milestone 2) . . . 40
6.15 CloudWatch monitoring (do after Milestone 2) . . . 42
6.16 Testing and validation (Milestone 1, task F) . . . 43
6.17 What persists and what changes . . . 43
6.18 Pausing between now and the interview . . . 44
6.19 Demo-day runbook: bring Milestone 1 back and show it . . . 44
6.20 Problems met during the real deployment . . . 46

Milestone 2 - containers on ECR and ECS Fargate . . . 47

7.1 The builder EC2 . . . 47

7.2 Copy the JARs and Dockerfiles (task A.I) . . . 48
7.3 Build the images (task A.II) . . . 48
7.4 Test the containers on the builder (task A.III) . . . 48
7.5 ECR: repositories, login, push (task B) . . . 49
7.6 Networking for ECS: namespace and security groups . . . 49
7.7 ECS cluster (task C.I) . . . 49
7.8 Task definitions (task C.II) . . . 50
7.9 The load balancer for the gateway . . . 50
7.10 ECS services (tasks C.III and C.IV) . . . 51
7.11 Frontend and RDS for Milestone 2 (task D) . . . 51
7.12 CloudWatch, testing and scaling (tasks E and F) . . . 52

Milestone 3 - GitHub . . . 53
8.1 What goes into Git . . . 53
8.2 Steps . . . 53

Troubleshooting, cost and clean-up . . . 54
9.1 Troubleshooting . . . 54
9.2 What costs money (approximate, us-east-1) . . . 55
9.3 Clean-up order . . . 55

Interview questions and answers . . . 56
Architecture and design . . . 56
Networking and security . . . 56
Services . . . 57

Remaining files, copy-paste ready . . . 58

Command cheat sheet . . . 60

PART 1

How to use this guide, and the big picture

What the case study asks, what you will build, the names used everywhere, and the order of work.

1.1 The case study in one page

Milestone	What must be delivered (from the case study)
1 - Manual deployment	React build on **S3** static website hosting; each Spring Boot microservice on its own **EC2** (Java installed, JAR deployed, security groups); **RDS MySQL** (schemas, users, services connected); **AWS Lambda** for an event-driven task triggered by **S3 uploads**, with permissions for S3 and RDS; **CloudWatch** logs and metrics for EC2 and Lambda (dashboards and alarms optional); end-to-end test.
2 - Containers	A **Dockerfile** per service, images built and tested; one **ECR** repository per service, Docker CLI authenticated, images pushed; an **ECS cluster on Fargate**, a task definition per service, service discovery, networking and security groups, ECS services; RDS through task environment variables; CloudWatch for tasks; end-to-end test.
3 - GitHub	Push the build to the GitHub repository (Deloitte GitHub Enterprise, per the guidelines).
Technical interview	Discussion of assumptions, functionalities and validations - Parts 3, 4 and 12 prepare you for it.

> **NOTE**
> Guidelines from the case study that shape every step: best coding practices; assumptions allowed; a simple working prototype beats a good-looking broken one; **comment the deployment steps and configuration**; **separate dev, test and prod environments**; **least-privilege IAM roles and security groups**; **regular database backups**; **Deloitte GitHub Enterprise only**; Git with pull requests. Best practices: clear resource naming, containerization, Eureka or AWS-native service discovery, Spring Cloud Gateway, CloudWatch, horizontal scaling, Thunder Client tests, Swagger docs.

1.2 What you will build - Milestone 1

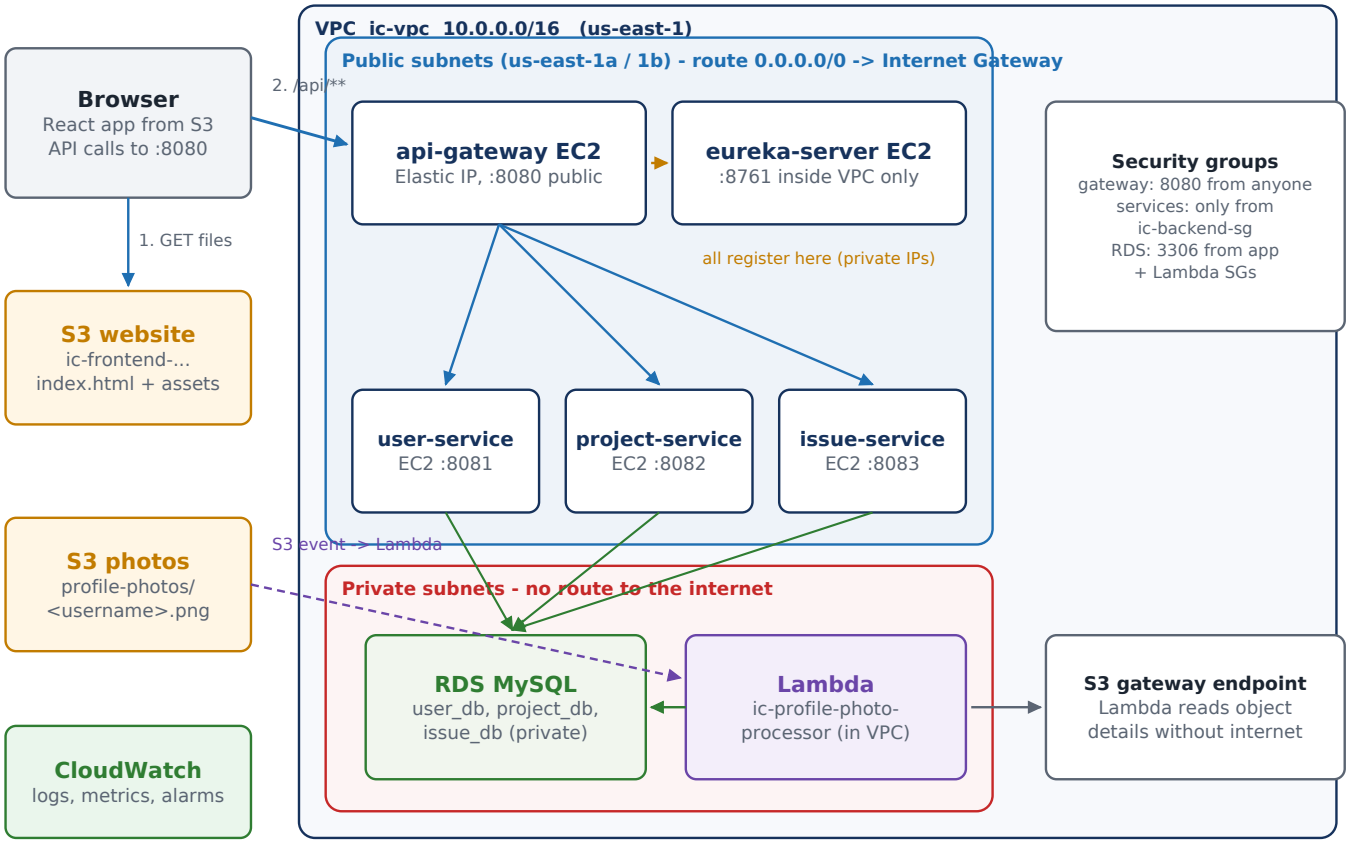


Figure 1.1 - Milestone 1. The browser loads the React files from S3 and calls the gateway on its Elastic IP. Everything else stays inside the VPC.

1.3 What you will build - Milestone 2

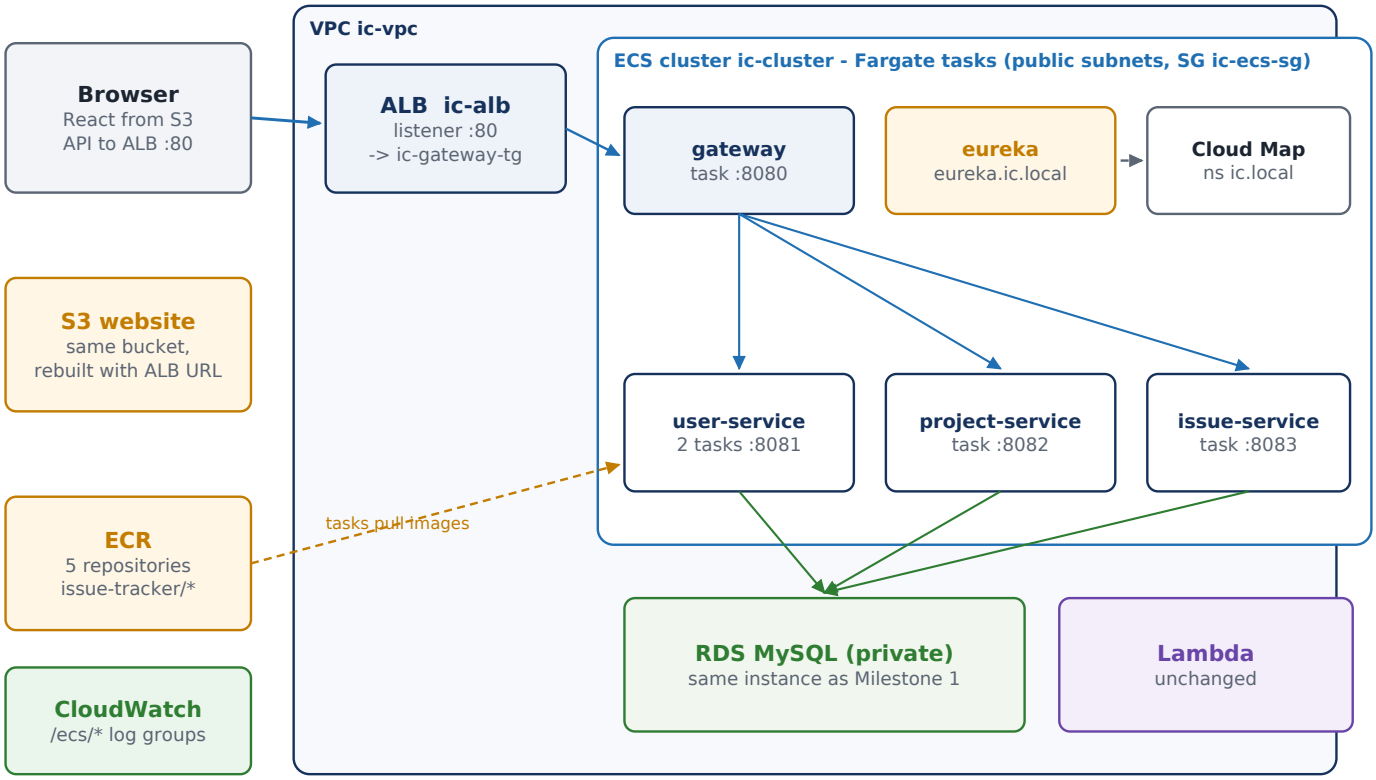


Figure 1.2 - Milestone 2. The same five JARs run as containers on Fargate. An Application Load Balancer is the public entry point; Eureka is found through Cloud Map DNS (eureka.ic.local).

1.4 Names used in this guide

Clear, consistent naming is a case-study best practice. Every resource starts with **ic** (InfinityCastle). Replace `<you>` with something unique such as your initials and a number - S3 bucket names are global across all AWS accounts.

Resource	Name
Region / AZs	`us-east-1` (N. Virginia), `us-east-1a`, `us-east-1b`
Key pair	`ic-key` (file `ic-key.pem`, kept outside the project folder)
VPC and subnets	`ic-vpc` (10.0.0.0/16), created by the "VPC and more" wizard with project name `ic`
Security groups	`ic-backend-sg`, `ic-gateway-public-sg`, `ic-rds-sg`, `ic-lambda-sg`, `ic-builder-sg`, `ic-alb-sg`, `ic-ecs-sg`
EC2 instances (M1)	`ic-eureka`, `ic-gateway`, `ic-user`, `ic-project`, `ic-issue`; builder (M2): `ic-builder`
IAM roles	`ic-ec2-cloudwatch-role`, `ic-builder-role`, `ic-photo-lambda-role`, `ecsTaskExecutionRole`
RDS	instance `ic-mysql`, schemas `user_db`, `project_db`, `issue_db`
S3 buckets	`ic-frontend-<you>` (website, M1), `ic-frontend-m2-<you>` (website, M2), `ic-photos-<you>` (profile photos); yours use `<you>` = `nlnu19`
Lambda	`ic-profile-photo-processor`
ECR repositories	`issue-tracker/eureka-server`, `.../api-gateway`, `.../user-service`, `.../project-service`, `.../issue-service`
ECS	cluster `ic-cluster`, task families `ic-<app>`, services `ic-<app>-svc`, namespace `ic.local`
Load balancer	`ic-alb`, target group `ic-gateway-tg`
CloudWatch	log groups `/issue-tracker/m1/<app>`, `/ecs/ic-<app>`, `/aws/lambda/...`; dashboard `ic-dashboard`

1.5 The order of work

1. **Part 5 - read the project changes** (already applied) and build the five JARs; check the app still runs on your laptop.
2. **Milestone 1 (Part 6):** worksheet -> laptop -> key pair -> VPC -> role -> security groups -> RDS -> ic-eureka (SSH test, deploy) -> the other four EC2s -> schemas and users -> services -> React on S3. **Done on 19 October 2026.** The Lambda, CloudWatch and the full test table (6.14-6.16) follow **after Milestone 2**.
3. **Milestone 2 (Part 7):** builder EC2 -> Docker images -> ECR -> Cloud Map -> ECS cluster -> task definitions -> load balancer -> services -> React rebuilt -> CloudWatch -> test -> scale.
4. **Milestone 3 (Part 8):** commit the changes and push to the GitHub repository.
5. **Between sessions:** what survives a stop/start is in **6.17**, how to pause in **6.18**, and the step-by-step demo-day start-up and demo script in **6.19**.
6. **Clean up (Part 9)** when you have finished the demo, so nothing keeps costing money.

> **WATCH OUT**
> Stop at every **Checkpoint** box. Each one tells you what you must see before going on. Most AWS problems come from a missed earlier step (a security group rule, a wrong environment variable); finding them at the checkpoint is much easier than at the end.

PART 2

The class notes - what was taught and how it all connects

Every topic from the 32 pages of notes, explained more fully, with the corrections the handwriting needs, and where each one is used in this case study.

2.1 The map: class demo -> case-study step

Class topic / demo	What you practised	Used in this case study
Cloud basics, regions, AZs	availability, scalability, elasticity, deployment and service models	choosing us-east-1 and two AZs; the design discussion (Part 4)
Virtualization, hypervisors, EC2 (Demo 1)	VMs, AMIs, key pairs, instance types, connecting	the five application servers and the builder
SSH, PuTTY, Instance Connect	`ssh -i key.pem ec2-user@host`, .ppk keys	every server task; SSH tunnels to private ports
httpd / Nginx (Demo 2)	install a server, open port 80 in the security group	the same pattern for port 8080
Spring Boot on EC2 (Demo 3)	Corretto 21, `mvn clean package`, `scp`, `java -jar`	Milestone 1 backend
VPC, subnets, IGW, routes, NAT, bastion (Demo 4)	networking from scratch	ic-vpc, public/private subnets, private RDS
Docker (Demos 6-7)	images, containers, `docker run -p`, Dockerfile	Milestone 2 images
ECR (Demo 8)	repository, `get-login-password`, tag, push, pull	Milestone 2 registry
ECS	cluster, task definition, task, service	Milestone 2 on Fargate
RDS + DBeaver	create MySQL, connect with endpoint, update application.properties	shared database (both milestones)
S3 static website	`npm run build`, bucket, policy, website hosting	React frontend

Not covered in the notes but required by the case study: **IAM roles** (needed by EC2, Lambda and ECS), **AWS Lambda**, **CloudWatch**, **Application Load Balancer** and **Cloud Map**. They are explained from scratch in Part 3.

2.2 Cloud fundamentals (notes page 1)

Term	Meaning	Example in our deployment
Availability	The system keeps answering even when a part fails	RDS backups; ECS restarts a crashed task automatically
Scalability	Can grow for long-term demand (bigger or more machines)	add a second user-service task (Part 7.12)
Elasticity	Grows and shrinks automatically with short-term demand	ECS service auto scaling on CPU (optional)
Load balancing	Spreads requests over several targets	ALB in front of the gateway; Spring Cloud LoadBalancer between services
Managed services	AWS runs the servers; you use the service	RDS, S3, Lambda, Fargate, ECR, CloudWatch

The load-balancer list in the notes reads "Application: ALB, Network: NLB, Database: DLB". AWS has no "DLB": the four Elastic Load Balancing types are **ALB** (Application, layer 7, HTTP routing), **NLB** (Network,

layer 4, TCP/UDP, very high throughput), **GWLB** (Gateway, for third-party firewall appliances) and the old **CLB** (Classic). The third item was most likely GWLB.

- **Deployment models:** public cloud (AWS), private cloud (your own data centre run like a cloud), hybrid (both, connected, e.g. with Direct Connect), multi-cloud (several providers).
- **Service models:** IaaS = you get machines and networks (EC2, VPC) and manage the OS; PaaS = you bring code or containers and the platform runs them (Elastic Beanstalk, Fargate, RDS for databases); SaaS = finished software (Gmail). FaaS/serverless (Lambda) is PaaS taken further: you bring a function.
- Milestone 1 is mostly **IaaS** (you patch and run the EC2s); Milestone 2 moves to **PaaS/CaaS** (Fargate runs the containers, you never see a server). Being able to say this is a good interview answer.

2.3 Global infrastructure (page 2)

A **Region** is a geographic area (us-east-1 = N. Virginia) with its own copy of most services. Each region has several **Availability Zones** - one or more separate data centres with their own power and network, close enough for fast links. **Edge locations** are small sites used by CloudFront (the CDN) to cache content near users. The notes quote 36 regions, 6 AZs in N. Virginia and 140+ edge locations; these numbers grow every year, so check the AWS global infrastructure page before quoting them. The unclear "240+" was most likely "245+ countries and territories served".

> **WHY TWO AZS**
> We use **two AZs** because RDS needs a subnet group covering at least two AZs and an ALB needs subnets in at least two AZs. Spreading across AZs is the basic high-availability pattern.

2.4 Virtualization and hypervisors (pages 2-4)

- A **virtual machine** has its own guest OS, binaries/libraries and application, and shares the physical server with other VMs through a **hypervisor**.
- **Type 1 (bare metal):** the hypervisor runs directly on the hardware - VMware ESXi, Microsoft Hyper-V, Xen, KVM, Oracle VM Server. AWS runs EC2 on its **Nitro** hypervisor (KVM-based, with networking and storage offloaded to dedicated cards).
- **Type 2 (hosted):** the hypervisor is an application on a normal OS - VirtualBox, VMware Workstation. Good for laptops, slower for servers.
- **EC2** gives you a VM ("instance") from an **AMI** (a disk image such as Amazon Linux 2023), an **instance type** (CPU/RAM size, e.g. t3.micro = 2 vCPU, 1 GiB), an **EBS volume** (the disk; 8 GB default in the notes) and a **key pair** for SSH.

2.5 Connecting and hosting (pages 4-11)

- `ssh -i "ic-key.pem" ec2-user@<public-DNS-or-IP>` - `ec2-user` is the default user of Amazon Linux. Windows 10/11 has OpenSSH built in, so this works in PowerShell; PuTTY needs the key converted to .ppk with PuTTYgen.
- **EC2 Instance Connect** opens a browser terminal without your key (it pushes a temporary key); it needs port 22 open to the Instance Connect service range, so "My IP" rules block it.
- **Private vs public IP:** the private IP (10.0.x.x) is used inside the VPC and never changes for the life of the instance; the public IP changes on every stop/start unless you attach an **Elastic IP**. That is why the gateway gets an Elastic IP: its address is compiled into the React build.
- The httpd and Nginx demos show the universal recipe: install, `systemctl enable` + `start`, open the port in the **security group**, browse to it. The notes mention changing httpd's `Listen 80` to another port (the handwriting looks like 5680): any port works if it is opened in the security group too.
- Demo 3 (Spring Boot): `sudo yum install -y java-21-amazon-corretto-devel`, build with `mvn clean package`, copy with `scp`, run with `java -jar`. Milestone 1 uses exactly this, plus systemd so the service

restarts by itself and survives a reboot. (`-headless` is enough on a server: it is the runtime without the desktop libraries; `-devel` adds the compiler.)

2.6 Networking (pages 12-18)

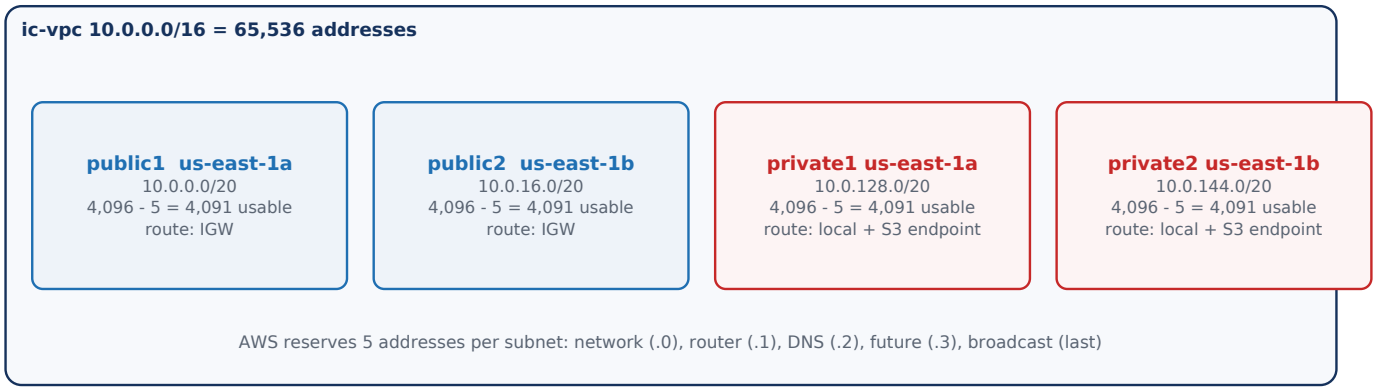


Figure 2.1 - The VPC the wizard builds for us: two public and two private /20 subnets in two AZs.

- **VPC** = your own isolated network in a region. **CIDR** (Classless Inter-Domain Routing) gives its address range: `10.0.0.0/16` means the first 16 bits are fixed, leaving 2^16 = 65,536 addresses.
- **Subnets** split the VPC and each lives in one AZ. A /20 has 2^12 = 4,096 addresses; AWS keeps 5, so 4,091 are usable. A subnet is **public** only because its **route table** sends `0.0.0.0/0` to an **Internet Gateway**; otherwise it is **private**.
- **NAT Gateway** (in a public subnet, with an Elastic IP) lets private resources reach the internet outbound only. It costs money per hour, so this guide avoids it: private resources here (RDS, Lambda) do not need the internet, and Lambda reaches S3 through a free **S3 gateway endpoint**.
- **VPC peering** joins two VPCs; **Transit Gateway** is a hub for many VPCs; **Direct Connect** is a private line from an office to AWS. Not needed here, but know the differences.
- **Bastion host (jump box):** a public instance used to reach private ones. The class copied the .pem onto the bastion; never do that outside a lab. Use `ssh -J ec2-user@<bastion> ec2-user@<private-ip>` (ProxyJump) - the key never leaves your laptop. In this guide `ic-gateway` doubles as the bastion for reaching RDS.

CORRECTIONS TO THE NOTES

Two values in the notes need care. (1) The VPC CIDR reads `12.35.0.0/16`, but the subnets are `10.35.16.0/20` and `10.35.32.0/20`. A subnet must lie inside its VPC, so the VPC must have been `10.35.0.0/16`. Always use private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) - 12.x.x.x belongs to real internet hosts. (2) The private IP `10.35.16.5` is plausible: the first usable address in 10.35.16.0/20 is .4, because AWS reserves .0 to .3.

2.7 Docker, ECR and ECS (pages 19-30)

- A **container** shares the host's kernel (no guest OS), so it starts in seconds and is small. An **image** is the read-only template built from a **Dockerfile**; a container is a running instance of it.
- `docker run -d -p 5566:80 nginx:latest` maps host port 5566 to container port 80. On **Fargate** there is no host port mapping: each task gets its own network card and IP (`awsvpc` mode), so the host port always equals the container port. The class example `http://<ip>:8989/api/hello` (8989 -> 8080) only works with `docker run` on an EC2.
- `docker ps` (running), `docker ps -a` (all), `docker images`, `docker stop`, `docker logs`. Without `sudo`, add `ec2-user` to the docker group (`sudo usermod -aG docker ec2-user`, then log in again).
- **ECR** is AWS's private Docker registry. `aws ecr get-login-password ... | docker login ...` gives Docker a token valid for 12 hours; then `docker tag` + `docker push`.

- **ECS** concepts from the notes: a **task definition** (blueprint: image, CPU, memory, ports, environment, logs), a **task** (a running copy), a **service** (keeps N tasks running, replaces failed ones, joins them to a load balancer) and a **cluster** (logical group). **Fargate** is the serverless launch type: no EC2 to manage.

2.8 RDS and S3 (pages 30-32)

- RDS: "Standard create" (the notes' "full configuration"), MySQL, Single-AZ, db.t3.micro, master user and password, VPC and security group, then connect with the **endpoint** (a DNS name such as `ic-mysql.xxxx.us-east-1.rds.amazonaws.com`) - in DBeaver and in `application.properties`.
- In class the database was made publicly accessible so DBeaver could reach it directly. Here RDS stays **private** (least privilege), and DBeaver connects through an **SSH tunnel** via `ic-gateway` (Part 6.9).
- S3 static website: `npm run build`, create a bucket, turn off "Block all public access", add a bucket policy allowing `s3:GetObject`, enable static website hosting with `index.html`. For a React app with React Router the **error document must also be** `index.html` (not error.html), otherwise refreshing `/owner/dashboard` returns a 404 from S3.
- The last page mentions uploading a WAR/JAR: in this case study the JARs go to EC2 (Milestone 1) or into Docker images (Milestone 2); S3 only holds the React files and the profile photos.

PART 3

AWS in depth - every service this deployment uses

What each service is, how it works underneath, the settings that matter, what it costs, and the mistakes to avoid.

3.1 The shared responsibility model

AWS is responsible for security **of** the cloud (buildings, hardware, hypervisor, the managed services' internals). You are responsible for security **in** the cloud: who can log in (IAM), which ports are open (security groups), patching your EC2 operating systems, your application code, your data and its encryption. The more managed the service, the less you own: on EC2 you patch Linux; on Fargate AWS does; on Lambda you only own the function code and its permissions.

3.2 IAM - who may do what

Concept	Meaning	Here
User	A person or program with long-term credentials (password, access keys)	your console login (given by the bootcamp)
Group	A set of users sharing permissions	-
Policy	JSON document: Effect (Allow/Deny), Action (`s3:GetObject`), Resource (an ARN), optional Condition	the Lambda's inline S3 policy
Role	An identity with permissions but no password; **assumed** by a service or person, which receives temporary credentials	EC2, Lambda and ECS roles
Instance profile	The container that attaches a role to an EC2 instance	created automatically with the role
Managed policy	Ready-made policy maintained by AWS	`CloudWatchAgentServerPolicy`, `AWSLambdaVPCAccessExecutionRole`
Trust policy	Who may assume a role (e.g. `lambda.amazonaws.com`)	set by the console when you pick the service

- **Least privilege** (a case-study guideline): grant only the actions and resources needed. The Lambda may only read objects under `profile-photos/` in one bucket - not "s3:*" on everything.
- **Roles instead of access keys on servers.** The class used `aws configure` with access keys on the VM. Keys stored on a disk can leak and never expire. An instance role gives the same permissions with temporary credentials that rotate automatically; the AWS CLI on the instance finds them by itself.
- Every action is checked: explicit Deny > Allow > implicit deny (anything not allowed is denied).
- **ARN** = Amazon Resource Name, e.g. `arn:aws:s3:::ic-photos-nk7/profile-photos/*`.

WATCH OUT
Bootcamp accounts are sometimes restricted (for example, you may not be allowed to create IAM roles). If a step fails with "not authorized", ask the trainer for an existing role with the listed policies, or use `aws configure` with temporary keys only on the builder and remove them afterwards.

3.3 EC2 in depth

Setting	What to know	Our choice
AMI	Disk image with the OS. Amazon Linux 2023 uses `dnf` (`yum` still works as an alias), has the AWS CLI v2 preinstalled, and its default user is `ec2-user`	Amazon Linux 2023
Instance type	Family + generation + size: t3 = burstable general purpose (earns CPU credits when idle). t3.micro 1 GiB, t3.small 2 GiB, t3.medium 4 GiB	t3.micro per app (t3.small if memory is short); builder t3.medium
EBS volume	Network disk, survives stop/start; gp3 is the default type	8 GiB gp3 (default)
Key pair	AWS keeps the public key; you download the private key once	`ic-key.pem`
Security group	Stateful virtual firewall on the network card (Part 3.5)	see the matrix in Part 4.4
IAM role	Permissions for software on the instance	`ic-ec2-cloudwatch-role`
User data	Script run once as root at first boot	installs Java, the CloudWatch agent and the service definition
Public IP / Elastic IP	Public IP changes on stop/start; Elastic IP is fixed until released. Every public IPv4 address is billed per hour	Elastic IP only on the gateway

Pricing models: On-Demand (per second, no commitment - what we use), Savings Plans / Reserved (1-3 year commitment, cheaper), Spot (spare capacity, up to ~90% cheaper, can be taken back with two minutes' notice). **Instance states:** stopped instances do not bill for compute but still bill for EBS and Elastic IPs; terminated instances are gone.

Running a JAR as a service with systemd. `java -jar` in an SSH session dies when you log out. A systemd unit (`/etc/systemd/system/issue-tracker.service`) makes Linux start the app at boot, restart it if it crashes (`Restart=on-failure`), run it as `ec2-user` (not root), load its settings from an environment file, and write its output to a log file that the CloudWatch agent ships. `SuccessExitStatus=143` tells systemd that the JVM's normal answer to a stop signal is not a failure.

3.4 VPC in depth

Piece	Role	In ic-vpc
CIDR block	Address range of the VPC	10.0.0.0/16
Subnet	Range inside one AZ	2 public (/20) + 2 private (/20)
Route table	Rules "destination -> target"; every subnet has exactly one	public: 0.0.0.0/0 -> IGW; private: local only + S3 endpoint
Internet Gateway	Two-way path to the internet for resources with public IPs	ic-igw
NAT Gateway	Outbound-only internet for private subnets (hourly cost)	not used
Gateway endpoint (S3, DynamoDB)	Private route to S3 without internet; free	used by the Lambda
Interface endpoint (PrivateLink)	Private IP for other AWS APIs (ECR, Logs, Secrets Manager); hourly cost	not used (tasks use public IPs instead)
DNS hostnames / resolution	Needed for RDS endpoints and Cloud Map names	enabled by the wizard
Network ACL	Stateless subnet firewall	default (allow all) - security groups do the work

3.5 Security groups vs network ACLs

	Security group	Network ACL
Applies to	a network card (instance, RDS, Lambda ENI, task)	a subnet
State	**stateful** - the answer to an allowed request is allowed automatically	**stateless** - return traffic needs its own rule
Rules	allow only	allow and deny, numbered, first match wins
Sources	IP ranges **or other security groups**	IP ranges only

Referencing a security group as the source is the key least-privilege technique in this guide: "RDS accepts 3306 from members of `ic-backend-sg`" stays correct when instances are added or their IPs change, and nothing outside the group can connect even if it knows the address.

3.6 S3 in depth

- **Buckets** hold **objects** identified by a **key** (`profile-photos/priya.png`). "Folders" in the console are just key prefixes. Bucket names are global and DNS-compatible.
- **Block Public Access** (4 switches, on by default) overrides any policy that would make objects public. For a website or public photos you turn off the two "bucket policy" switches (or all four, as in class).
- **Bucket policy** = resource-based IAM policy on the bucket. `"Principal": "*"` + `s3:GetObject` on `arn:aws:s3:::bucket/*` makes objects publicly readable (not listable, not writable).
- **Static website hosting** gives an HTTP endpoint such as `http://ic-frontend-nk7.s3-website-us-east-1.amazonaws.com` (some regions use `s3-website.<region>`; always copy it from the console). It serves `index.html` for `/` and the error document for missing keys. It is **HTTP only** - HTTPS needs CloudFront in front.
- **Event notifications** send "object created/removed" events to Lambda, SQS or SNS, filtered by prefix and suffix. Delivery is asynchronous and at-least-once (very rarely twice), so the Lambda must be **idempotent** - running it twice must do no harm. Ours is: it sets the same URL again.
- **Versioning** keeps old versions of overwritten objects (a cheap backup). Storage classes (Standard, Intelligent-Tiering, Glacier) trade price for access speed.

3.7 RDS in depth

Feature	Meaning	Our setting
Engine / version	MySQL 8.x managed by AWS (patching, backups, monitoring)	MySQL 8.0 or 8.4 (latest offered)
Instance class	CPU/RAM of the DB server	db.t3.micro / db.t4g.micro
Storage	gp2/gp3 SSD; can auto-grow	20 GiB gp3
Single-AZ vs Multi-AZ	Multi-AZ keeps a synchronous standby in another AZ with automatic failover (high availability, double cost)	Single-AZ (as in class); say Multi-AZ for production
Read replica	Asynchronous copy for read scaling (not failover)	not used
DB subnet group	Which subnets (>= 2 AZs) RDS may use	the two private subnets
Public access	Whether RDS gets a public IP	**No**
Automated backups	Daily snapshot + transaction logs -> restore to any second in the retention window	7 days (case study: regular backups)
Manual snapshot	Kept until you delete it	take one before the demo
Endpoint	DNS name to connect to; never use the IP (it changes on failover)	in `DB_HOST`
Parameter group	Server settings (e.g. `require_secure_transport`)	default
Encryption at rest	KMS-encrypted storage and snapshots	enabled (default)

WHY ONE RDS INSTANCE FOR THREE SERVICES

One RDS instance hosts the three schemas because it keeps cost and set-up small, while **each service still owns its own database and its own MySQL user** that can only touch that schema. That preserves the "database per service" rule of microservices at the logical level. In production you could give each service its own instance.

3.8 AWS Lambda in depth

- **Serverless functions:** you upload code with a **handler** (`lambda_function.lambda_handler`); AWS runs it when an **event** arrives and bills per request and per millisecond of run time. No server, no idle cost.
- **Event sources:** S3 (ours), API Gateway, SQS, EventBridge schedules, DynamoDB streams... S3 calls the function **asynchronously**; failed attempts are retried twice.
- **Execution role:** the permissions the code runs with. Ours: `AWSLambdaVPCAccessExecutionRole` (write CloudWatch logs + create network cards in the VPC) and one inline policy for `s3:GetObject` on the photos prefix. **Resource-based policy:** allows S3 to invoke the function - the console adds it when you create the trigger.
- **Lifecycle:** the first call creates an execution environment (a **cold start**: loading the runtime and your code); later calls reuse it (**warm**). Code outside the handler (the S3 client) runs once per environment, so it is the place for expensive set-up.
- **Settings:** memory 128 MB-10 GB (CPU grows with memory), timeout up to 15 min (ours 15 s), environment variables (DB host, user, password), runtime (Python 3.12).
- **In a VPC:** to reach the private RDS the function is attached to the private subnets with `ic-lambda-sg`. It then has **no internet access**; it reaches S3 through the S3 gateway endpoint. Libraries not in the runtime (PyMySQL) are shipped **inside the .zip** (or a layer).
- **Logs:** everything printed or logged goes to the log group `/aws/lambda/<function-name>` automatically.

3.9 CloudWatch in depth

Feature	Meaning	Here
Metrics	Numbers over time per namespace and dimension. EC2 sends CPU, network, disk I/O by default; memory and disk usage need the **CloudWatch agent**	CPUUtilization, mem_used_percent, Lambda Errors, ECS CPU/Memory
Logs	Log groups (per app) contain log streams (per instance/task); retention in days	/issue-tracker/m1/, */aws/lambda/*, /ecs/*
Logs Insights	Query language over log groups	find errors across all services
Alarms	Watch one metric against a threshold; actions such as an SNS e-mail	CPU > 70%, Lambda errors >= 1
Dashboards	Graphs of chosen metrics on one page	`ic-dashboard`
Container Insights	Extra task/service metrics for ECS	enabled on the cluster (optional)

3.10 Docker in depth

- An **image** is a stack of read-only **layers**; each Dockerfile instruction (`FROM`, `COPY`, `RUN`) adds one. Layers are cached and shared, so rebuilding after changing only the JAR is fast.
- Our Dockerfile: `FROM amazoncorretto:21-alpine` (Java runtime on a small Linux), create a non-root user, `COPY` the JAR, `EXPOSE` the port (documentation only), `ENTRYPOINT` starting Java. `-XX:MaxRAMPercentage=75` makes the JVM size its heap from the container's memory limit.
- **Tags** name versions: `user-service:1.0.0`; the full ECR name adds the registry: `<account>.dkr.ecr.us-east-1.amazonaws.com/issue-tracker/user-service:1.0.0`. Prefer version tags over `latest` so you always know what is running.
- **Networks:** containers on the same user-defined network (`docker network create ic-net`) reach each other by container name - we use that to test all five images on the builder before ECR.
- **CPU architecture:** an image built on an x86_64 EC2 (t3) runs on Fargate X86_64; a Graviton (t4g/ARM) build needs ARM64 tasks.

3.11 ECR and ECS / Fargate in depth

Concept	Meaning
ECR repository	Private image store, one per service; "scan on push" checks for known vulnerabilities; lifecycle policies delete old images
Cluster	Logical group of services/tasks; with Fargate there are no instances to manage
Task definition	Versioned blueprint (revisions 1, 2, 3...): image, CPU/memory (e.g. 0.5 vCPU / 1 GB), port mappings, environment variables, secrets, log configuration, roles
Task execution role	Used by ECS itself to pull from ECR and write logs (`ecsTaskExecutionRole`)
Task role	Used by your code to call AWS APIs (not needed: our services only call each other and MySQL)
awsvpc network mode	Each task gets its own network card, private IP and security group (required by Fargate)
Service	Keeps the desired number of tasks running, replaces unhealthy ones, rolling deployments, registers tasks in a target group and/or Cloud Map
Health check grace period	Seconds to ignore load-balancer health checks after a task starts (Spring Boot needs ~60-90 s on 0.5 vCPU)
Service auto scaling	Target tracking, e.g. keep average CPU at 60% between 1 and 3 tasks
Fargate vs EC2 launch type	Fargate: per-task billing, no servers; EC2 launch type: you run the container hosts (cheaper at scale, more work)

3.12 Load balancing and service discovery

- **Application Load Balancer** (layer 7): a stable DNS name in two or more AZs; **listeners** (port 80) with **rules** forward to **target groups**; each target is checked with a **health check** (our gateway: `GET /actuator/health` must return 200). Targets of type **IP** are needed for Fargate tasks.
- **Two levels of load balancing in our design:** the ALB spreads browser traffic over gateway tasks; inside, Spring Cloud LoadBalancer spreads gateway and Feign calls over the instances Eureka knows (two user-service tasks -> round robin).
- **Service discovery** answers "where is service X now?". Our apps already use **Eureka** (client-side discovery: services register their IP:port and fetch the registry). Only Eureka itself needs a fixed address: on EC2 its private IP; on ECS a **Cloud Map** private DNS name `eureka.ic.local` that ECS keeps pointing at the current Eureka task. AWS-native alternatives: Cloud Map for every service, or ECS Service Connect.
- **Why Eureka must register IPs on AWS:** the current config says `eureka.instance.hostname=localhost`, so every service would tell Eureka "I am at localhost", and the gateway would call itself. `prefer-ip-address=true` makes each instance register its private IP instead (Part 5.2).

3.13 The Well-Architected Framework (for the interview)

Pillar	How this deployment addresses it
Operational excellence	documented runbook, systemd, CloudWatch logs and alarms, repeatable Docker images
Security	least-privilege IAM roles, security groups referencing each other, private RDS, per-service DB users, no keys in Git, TLS to MySQL
Reliability	systemd/ECS restart failed processes, RDS automated backups, two AZs, ALB health checks
Performance efficiency	right-sized instances, horizontal scaling of tasks, front-end filtering, parallel requests in the React app
Cost optimisation	small instances, no NAT gateway, S3 for static files, clean-up checklist
Sustainability	managed services and right-sizing avoid idle capacity

PART 4

Design decisions, security plan and environments

Every important choice with its alternatives and the reason - the answers to "why did you do it this way?".

4.1 Decisions and their reasons

Decision	Alternatives	Reason
One EC2 per application (M1)	all on one big EC2; two EC2s	matches "provision EC2 instances for each microservice", each app scales and fails independently, and Eureka/Feign are exercised over a real network
Apps in public subnets with strict SGs (M1)	private subnets + NAT + bastion	no NAT cost; only the gateway port is open to the world, everything else only from `ic-backend-sg` or your IP
RDS private, one instance, three schemas	public RDS; one RDS per service	least exposure; low cost; per-service users keep isolation
Configuration via environment variables + Spring profile `aws`	edit application.properties on each server; separate JARs per environment	one JAR for every environment (12-factor), secrets stay out of the code and Git
Elastic IP on the gateway (M1)	raw public IP; ALB already in M1	the React build contains the API address, so it must not change
Lambda = profile-photo processor	CSV import, attachment log, scheduled report	fits the case-study diagram (S3 upload -> Lambda -> RDS) and our existing photo/initials feature
Lambda in Python with PyMySQL	Java Lambda	a tiny function; quick cold start; simple packaging
S3 gateway endpoint for Lambda	NAT gateway	free, and keeps the Lambda off the internet
Column-level DB grant for Lambda	reuse user_svc	Lambda can only read `username` and change `profile` - nothing else
Docker build on a builder EC2	Docker Desktop on the laptop	exactly the class workflow; no local installs; x86 images
Keep Eureka in ECS + Cloud Map for Eureka	replace Eureka by Cloud Map/Service Connect	no code change; the same discovery model as Milestone 1; Cloud Map gives Eureka a stable name
ALB in front of the gateway (M2)	public IP on the gateway task	task IPs change on every deployment; ALB gives a fixed DNS name, health checks, scaling
Fargate tasks in public subnets with public IPs	private subnets + NAT or VPC endpoints	tasks must pull images from ECR; this avoids NAT/endpoint costs; inbound is still blocked by `ic-ecs-sg`
HTTP only	HTTPS with ACM certificates + CloudFront/ALB	no domain name in the bootcamp; noted as the first production improvement

4.2 How a request travels

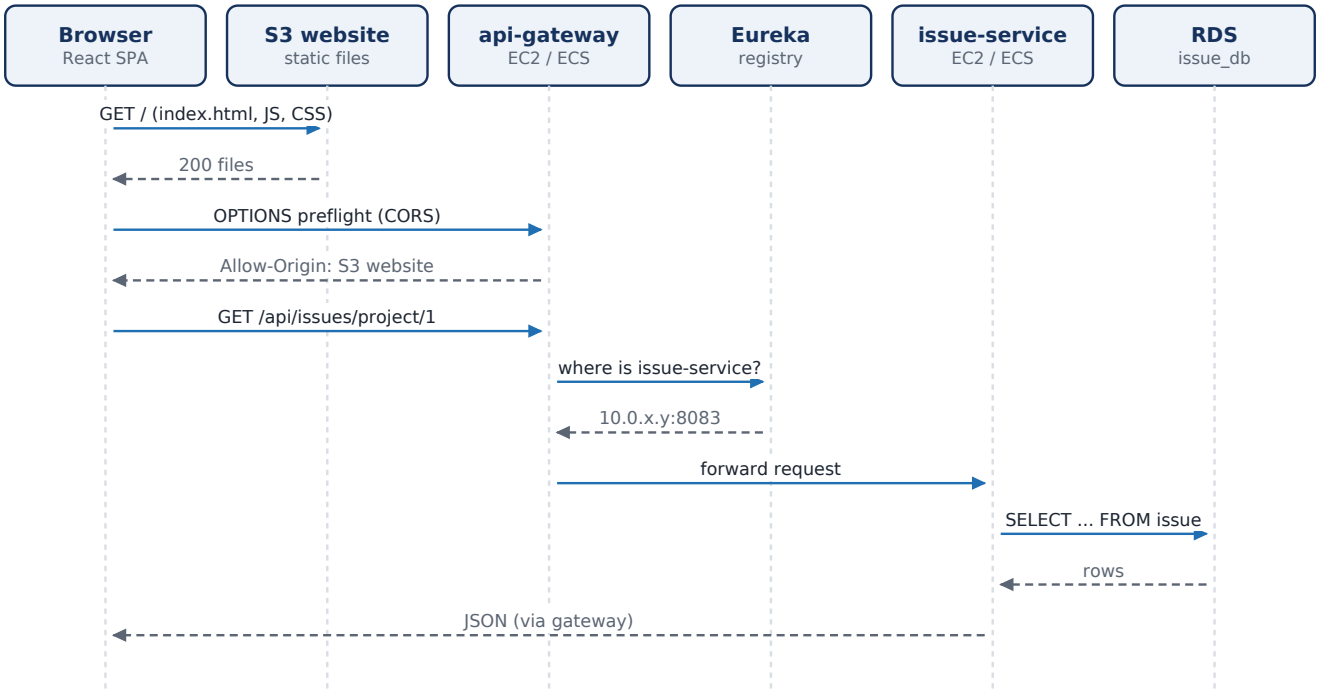


Figure 4.1 - A board load in Milestone 1. In Milestone 2 the browser calls the ALB, which forwards to a gateway task; the rest is identical.

4.3 Least-privilege IAM

Role	Trusted by	Permissions	Why
`ic-ec2-cloudwatch-role`	EC2	`CloudWatchAgentServerPolicy` (+ optional `AmazonSSMManagedInstanceCore`)	the agent sends logs and memory/disk metrics; SSM allows Session Manager
`ic-builder-role`	EC2	`AmazonEC2ContainerRegistryPowerUser`	push images to ECR without access keys
`ic-photo-lambda-role`	Lambda	`AWSLambdaVPCAccessExecutionRole` + inline `s3:GetObject` on `ic-photos-<you>/profile-photos/*`	logs, VPC network cards, read object details only
`ecsTaskExecutionRole`	ECS tasks	`AmazonECSTaskExecutionRolePolicy`	pull images, write logs

4.4 Security-group matrix

Security group	Inbound rules (type, port, source)	Attached to
`ic-backend-sg`	SSH 22 from **0.0.0.0/0** (key-only login; "My IP" fails behind the company proxy); TCP 8761, 8080-8083 from **ic-backend-sg** (itself)	all five M1 EC2s
`ic-gateway-public-sg`	TCP 8080 from 0.0.0.0/0	`ic-gateway` only
`ic-rds-sg`	MySQL 3306 from **ic-backend-sg**, **ic-lambda-sg**, **ic-builder-sg**, **ic-ecs-sg**	RDS
`ic-lambda-sg`	none (outbound only)	the Lambda
`ic-builder-sg`	SSH 22 from 0.0.0.0/0	`ic-builder`
`ic-alb-sg`	HTTP 80 from 0.0.0.0/0	the ALB
`ic-ecs-sg`	TCP 8080 from **ic-alb-sg**; TCP 8761, 8080-8083 from **ic-ecs-sg** (itself)	all ECS tasks

> **NOTE**
> Outbound rules stay at the default (all traffic allowed): the apps must reach Eureka, each other, RDS, and (on EC2) the package repositories. Because security groups are stateful, answers to allowed inbound requests are always let back out.

4.5 Secrets

- Passwords live only in: your password manager; `/etc/issue-tracker/app.env` on each EC2 (owner root, mode 600); Lambda and ECS environment variables. **Never** in Git, the React build, screenshots or chat.
- Better (optional): store each password in **Secrets Manager** or **SSM Parameter Store** (SecureString) and reference it from the ECS task definition ("ValueFrom") - the password then never appears in the console. The execution role needs permission to read that one secret.
- Your local `db.properties` is packaged inside the JARs you build (it is in `src/main/resources`). It only contains your laptop's MySQL password, but do not share the JARs publicly. The `aws` profile overrides it on AWS.

4.6 Environments: dev, test, prod (case-study guideline)

Environment	Where	How it is selected
dev	your laptop: local MySQL, Eureka on localhost, Vite on 5173	default Spring profile; `.env.development` / defaults
test	same AWS design with `ic-test-*` names, its own schemas or RDS instance	profile `aws` + test variables
prod	the deployment in this guide	profile `aws` + prod variables; React built with `.env.production`

The code is identical in every environment; only configuration differs. That is the point of putting addresses and passwords in environment variables. Name every AWS resource with its environment so nothing is mixed up (for a second environment, `ic-test-mysql` next to `ic-mysql`).

4.7 Backups

- RDS automated backups with 7-day retention (point-in-time restore) + a manual snapshot before demos and before deleting the instance.
- S3 versioning on both buckets (overwritten photos or a bad frontend upload can be restored).
- Code and configuration templates in Git; images in ECR with version tags (roll back by deploying an older task-definition revision).

4.8 Authentication (non-functional requirement)

The application checks email and password at login, but the API does not yet verify who sends each request (there is no token). The deployment keeps internal services private, so only the gateway is reachable. The next step would be JWT: the user-service issues a signed token at login, the React app sends `Authorization: Bearer <token>`, and the gateway validates it before routing. On AWS, **Amazon Cognito** could provide sign-up/sign-in and tokens, and an ALB can authenticate users with Cognito before forwarding. Name this as a known limitation and the planned improvement.

PART 5

The changes in the project

Every file that was changed or added for AWS, where it is, its complete content and the reason. These changes are ALREADY APPLIED in your project (18 October 2026) - read this part to understand them, then build the JARs (5.9).

STATUS
Already done in issue-tracking-system: the four `application-aws.properties` files, the gateway CORS property and `GatewayConfig` change, `optional:` db.properties, five Dockerfiles, the `aws/` folder (scripts, SQL, Lambda, helper scripts) and `.gitignore`. Checked: Java syntax, the Lambda against eight test uploads, shell syntax, the React build and lint. Your part: the five Maven builds in Spring Tools (5.9) and a quick local test.

5.1 Overview

#	File (inside issue-tracking-system)	Change	Why
1	`user-service`, `project-service`, `issue-service`: `src/main/resources/application-aws.properties`	new	Eureka by IP + RDS from environment variables
2	`api-gateway/src/main/resources/application-aws.properties`	new	Eureka by IP + CORS origin from a variable
3	`api-gateway/src/main/resources/application.properties`	add 1 property	local CORS origins as a property
4	`api-gateway/.../config/GatewayConfig.java`	edit `corsWebFilter`	read the origins instead of a fixed list
5	the three services' `application.properties`	`optional:` in front of `classpath:db.properties`	start even when no db.properties exists
6	`aws/scripts/build-frontend.ps1`	new	builds the React app with the AWS API address
7	`<app>/Dockerfile` (5 files)	new	Milestone 2 images
8	`aws/` folder: `ec2/user-data.sh`, `ec2/app.env.example`, `rds/*.sql`, `lambda/...`	new	deployment scripts, documented and versioned (case-study guideline)
9	root `.gitignore`	add 4 lines	keys, real settings files and build zips never reach Git

NOTE
eureka-server needs no change: it does not register with itself, and the other apps reach it by IP or DNS name. **No Java code changes in the three services.** The only Java change is in the gateway's CORS configuration.

5.2 The `aws` Spring profile for the three services

Create `src/main/resources/application-aws.properties` next to `application.properties` in each service, with the schema name of that service. Spring Boot loads `application.properties` first, then `application-aws.properties` when the profile `aws` is active, and the profile file wins. On the laptop nothing is active, so everything works exactly as before.

Listing 5.1 - `user-service/src/main/resources/application-aws.properties`

```
# ================================================================================================
# AWS profile of user-service - loaded in addition to application.properties when SPRING_PROFILES_ACTIVE=aws.
# Every value that differs between the laptop and AWS comes from an environment variable, so the SAME
# JAR runs locally (default profile), on EC2 (Milestone 1) and on ECS Fargate (Milestone 2).
# ================================================================================================

# Eureka registry address, e.g. http://10.0.3.25:8761/eureka/ (EC2) or http://eureka.ic.local:8761/eureka/ (ECS)
eureka.client.service-url.defaultZone=${EUREKA_URL}
# Register this instance by its private IP instead of "localhost", so other machines can call it
eureka.instance.prefer-ip-address=true
# Unique per running copy - two tasks of the same service must not share an instance id
eureka.instance.instance-id=${spring.application.name}:${spring.cloud.client.ip-address}:${server.port}

# Amazon RDS (MySQL). DB_HOST = RDS endpoint. The schema is created by 01-schemas-and-users.sql.
# sslMode=REQUIRED: the connection to MySQL is encrypted.
spring.datasource.url=jdbc:mysql://${DB_HOST}:3306/user_db?sslMode=REQUIRED
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

`project-service` and `issue-service`: the same file, with `project-service` / `project_db` and `issue-service` / `issue_db` in the two marked places (Appendix A has all three).

- `${EUREKA_URL}` - a placeholder: Spring replaces it with the environment variable of that name. If the variable is missing the app fails at start-up with a clear message - better than silently using localhost.
- `spring.cloud.client.ip-address` - provided by Spring Cloud: the machine's (or task's) own IP.
- `sslMode=REQUIRED` replaces the local `useSSL=false`: traffic to RDS is encrypted. `createDatabaseIfNotExist` is dropped because the service users are not allowed to create schemas.

5.3 The gateway: CORS from configuration

Listing 5.2 - `api-gateway/src/main/resources/application-aws.properties`

```
# ================================================================================================
# AWS profile of api-gateway - loaded when SPRING_PROFILES_ACTIVE=aws.
# ================================================================================================
eureka.client.service-url.defaultZone=${EUREKA_URL}
eureka.instance.prefer-ip-address=true
eureka.instance.instance-id=${spring.application.name}:${spring.cloud.client.ip-address}:${server.port}

# The React app's address on S3 (copied from the bucket's static website hosting panel), e.g.
# http://ic-frontend-nk7.s3-website-us-east-1.amazonaws.com  - read by GatewayConfig for CORS
app.cors.allowed-origins=${FRONTEND_ORIGIN}
```

Add these lines to the end of `api-gateway/src/main/resources/application.properties`:

```
# Origins (scheme://host:port) allowed to call the API from a browser - read by GatewayConfig.
# The aws profile replaces this list with the S3 website address (FRONTEND_ORIGIN).
app.cors.allowed-origins=http://localhost:5173,http://localhost:4173,http://localhost:3000,\
  http://localhost:8081,http://localhost:8082,http://localhost:8083
```

In `GatewayConfig.java`, change the `corsWebFilter` bean so it receives the list (add the import `org.springframework.beans.factory.annotation.Value`):

```
    @Bean
    public CorsWebFilter corsWebFilter(@Value("${app.cors.allowed-origins}") String[] allowedOrigins) {
        CorsConfiguration config = new CorsConfiguration();
        // Laptop: the localhost addresses from application.properties.
        // AWS: the S3 website address (FRONTEND_ORIGIN, see application-aws.properties).
        // No code change between them.
        config.setAllowedOrigins(List.of(allowedOrigins));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("*"));
        config.setMaxAge(3600L);
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return new CorsWebFilter(source);
    }
```

DETAILS

Spring turns a comma-separated value into a `String[]` automatically. The trailing `\` in the properties file continues the value on the next line. The origin must match the browser's address bar exactly (scheme, host, port, no trailing slash).

5.4 Optional db.properties

In the three services' `application.properties` change `spring.config.import=classpath:db.properties` to `spring.config.import=optional:classpath:db.properties`. Without `optional:` an app built on a machine that has no `db.properties` (a fresh clone, a CI build) would refuse to start, even though the `aws` profile supplies the credentials.

5.5 The React build for AWS

Vite bakes `VITE_API_BASE_URL` into the JavaScript at build time. A variable that already exists in the environment wins over the `.env` files, so `aws/scripts/build-frontend.ps1` sets it only for one build and removes it again - no file to edit, and local `npm run dev` keeps using localhost:

```
.\aws\scripts\build-frontend.ps1 -ApiUrl http://<GATEWAY-ELASTIC-IP>:8080/api     # Milestone 1
.\aws\scripts\build-frontend.ps1 -ApiUrl http://<ALB-DNS-NAME>/api                # Milestone 2 (port 80)
```

(A `.env.production` file would work too, but the remote tools are not allowed to write `.env` files to your laptop, and an environment variable leaves nothing to forget in Git.)

5.6 Dockerfiles (Milestone 2)

Listing 5.3 - `user-service/Dockerfile` (no extension)

```
# Container image of user-service (Milestone 2).
# Build context = the user-service folder, after "mvn clean package" created target/user-service-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/user-service-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8081

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

Folder	Changes from Listing 5.3
`eureka-server/Dockerfile`	name `eureka-server`, `EXPOSE 8761`
`api-gateway/Dockerfile`	name `api-gateway`, `EXPOSE 8080`

Folder	Changes from Listing 5.3
project-service/Dockerfile	name project-service, EXPOSE 8082
issue-service/Dockerfile	name issue-service, EXPOSE 8083

5.7 The aws/ folder - deployment scripts

```
issue-tracking-system/
  aws/
    ec2/user-data.sh                    Listing 5.4 - first-boot script for each application EC2
    ec2/app.env.example                 Listing 5.5 - template of /etc/issue-tracker/app.env
    rds/01-schemas-and-users.sql        Listing 5.6
    rds/02-lambda-user.sql              Listing 5.7
    lambda/profile-photo-processor/
      lambda_function.py                Listing 5.8
      requirements.txt                  one line:  pymysql>=1.1,<2
      package-lambda.sh                 builds photo-lambda.zip (run on ic-gateway)
    scripts/deploy-jar.ps1              laptop: copy one JAR to its EC2 + restart (Milestone 1)
    scripts/build-frontend.ps1          laptop: React build with the AWS API address
    scripts/copy-to-builder.ps1         laptop: Dockerfiles + JARs to ic-builder (Milestone 2)
    scripts/build-and-push.sh           ic-builder: build the 5 images and push them to ECR
    README.md                           what each file is for
```

Listing 5.4 - aws/ec2/user-data.sh

```
#!/bin/bash
# ================================================================================================
# EC2 user data for ONE Issue Tracker application server (Milestone 1).
# Runs once, as root, on first boot. Change SERVICE (and nothing else) for each of the five instances:
#   eureka-server | api-gateway | user-service | project-service | issue-service
# It installs Java 21, the MySQL client and the CloudWatch agent, prepares folders, the systemd service
# and the CloudWatch agent configuration. The JAR and the settings file are added later (Part 6.7).
# ================================================================================================
SERVICE=user-service
JAR=${SERVICE}-1.0.0.jar

# 1. Software
dnf install -y java-21-amazon-corretto-headless mariadb105 amazon-cloudwatch-agent

# 2. Folders: the JAR, the settings (secret, root only) and the log file
mkdir -p /home/ec2-user/app /etc/issue-tracker /var/log/issue-tracker
chown ec2-user:ec2-user /home/ec2-user/app
touch /etc/issue-tracker/app.env
chmod 600 /etc/issue-tracker/app.env

# 3. systemd service: start at boot, restart on failure, run as ec2-user, log to a file
cat > /etc/systemd/system/issue-tracker.service <<EOF
[Unit]
Description=Issue Tracker - ${SERVICE}
After=network-online.target
Wants=network-online.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/app
EnvironmentFile=/etc/issue-tracker/app.env
ExecStart=/usr/bin/java -Xms128m -Xmx384m -jar /home/ec2-user/app/${JAR}
SuccessExitStatus=143
Restart=on-failure
RestartSec=10
StandardOutput=append:/var/log/issue-tracker/app.log
StandardError=append:/var/log/issue-tracker/app.log

[Install]
WantedBy=multi-user.target
EOF

# 4. CloudWatch agent: ship the log file + memory and disk usage (the \$ keeps ${aws:...} for the agent)
cat > /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json <<EOF
{
  "agent": { "run_as_user": "root" },
  "logs": { "logs_collected": { "files": { "collect_list": [
    { "file_path": "/var/log/issue-tracker/app.log",
      "log_group_name": "/issue-tracker/m1/${SERVICE}",
      "log_stream_name": "{instance_id}",
      "retention_in_days": 14 } ] } } },
```

```
  "metrics": {
    "namespace": "IssueTracker/EC2",
    "append_dimensions": { "InstanceId": "\${aws:InstanceId}" },
    "metrics_collected": {
      "mem":  { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] } } }
}
EOF
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -s \
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json

# 5. Register the service (it is started by hand once the JAR and app.env are in place)
systemctl daemon-reload
systemctl enable issue-tracker
```

Listing 5.5 - aws/ec2/app.env.example

```
# Template of /etc/issue-tracker/app.env on each application EC2 (owner root, chmod 600).
# The real file holds passwords: create it on the server only, NEVER commit it.
SPRING_PROFILES_ACTIVE=aws
EUREKA_URL=http://<EUREKA-PRIVATE-IP>:8761/eureka/
DB_HOST=<RDS-ENDPOINT>
DB_USERNAME=user_svc
DB_PASSWORD=<user_svc password>
# api-gateway only (no DB_ lines on the gateway):
# FRONTEND_ORIGIN=http://<bucket>.s3-website-us-east-1.amazonaws.com
# eureka-server: leave the file empty
```

Listing 5.6 - aws/rds/01-schemas-and-users.sql

```
-- ============================================================================
-- 01-schemas-and-users.sql - run ONCE as the RDS master user, from ic-gateway:
--   mysql -h <RDS-ENDPOINT> -u admin -p --ssl < 01-schemas-and-users.sql
-- One schema per microservice and one MySQL user per service that can use only its own schema.
-- '10.0.%' = connections from inside ic-vpc only. Replace the three passwords before running.
-- ============================================================================
CREATE DATABASE IF NOT EXISTS user_db;
CREATE DATABASE IF NOT EXISTS project_db;
CREATE DATABASE IF NOT EXISTS issue_db;

CREATE USER IF NOT EXISTS 'user_svc'@'10.0.%'    IDENTIFIED BY 'CHANGE_ME_User#2026';
CREATE USER IF NOT EXISTS 'project_svc'@'10.0.%' IDENTIFIED BY 'CHANGE_ME_Project#2026';
CREATE USER IF NOT EXISTS 'issue_svc'@'10.0.%'   IDENTIFIED BY 'CHANGE_ME_Issue#2026';

-- Hibernate (ddl-auto=update) needs CREATE/ALTER/INDEX/REFERENCES to build its tables. No DROP, no other
» schema.
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, REFERENCES ON user_db.*    TO 'user_svc'@'10.0.%';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, REFERENCES ON project_db.* TO
» 'project_svc'@'10.0.%';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, REFERENCES ON issue_db.*   TO 'issue_svc'@'10.0.%';

SHOW DATABASES;
SELECT user, host FROM mysql.user WHERE user LIKE '%svc';
```

Listing 5.7 - aws/rds/02-lambda-user.sql

```
-- ============================================================================
-- 02-lambda-user.sql - run AFTER user-service has started once (its tables must exist):
--   mysql -h <RDS-ENDPOINT> -u admin -p --ssl < 02-lambda-user.sql
-- Column-level grant: the Lambda may read user names and change the profile column - nothing else.
-- ============================================================================
CREATE USER IF NOT EXISTS 'photo_lambda'@'10.0.%' IDENTIFIED BY 'CHANGE_ME_Lambda#2026';
GRANT SELECT (username), UPDATE (profile) ON user_db.`user` TO 'photo_lambda'@'10.0.%';
SHOW GRANTS FOR 'photo_lambda'@'10.0.%';
```

Listing 5.8 - aws/lambda/profile-photo-processor/lambda_function.py

```
"""
InfinityCastle - profile photo processor (AWS Lambda, Python 3.12).

Trigger : S3 "All object create events" on the photos bucket, prefix "profile-photos/".
Input   : uploaded files named profile-photos/<username>.<jpg|jpeg|png|gif|webp>
Work    : 1. check the key, the extension and the size (max 2 MB)
          2. ask S3 for the object's Content-Type (must be image/...)
          3. save the photo's public URL as that user's profile in RDS (user_db.user.profile)
Output  : a status per file; every decision is logged to CloudWatch Logs.

Environment variables: DB_HOST (RDS endpoint), DB_NAME (user_db), DB_USER (photo_lambda), DB_PASSWORD.
global-bundle.pem (the Amazon RDS certificates) is packaged next to this file, for TLS to MySQL.
"""
```

```
import logging
import os
import urllib.parse

import boto3
import pymysql
from pymysql.constants import CLIENT

logger = logging.getLogger()
logger.setLevel(logging.INFO)

PREFIX = "profile-photos/"
ALLOWED_EXTENSIONS = {"jpg", "jpeg", "png", "gif", "webp"}
MAX_SIZE_BYTES = 2 * 1024 * 1024
CA_BUNDLE = os.path.join(os.path.dirname(__file__), "global-bundle.pem")

# Created once per execution environment and reused by warm invocations
s3 = boto3.client("s3")


def lambda_handler(event, context):
    """Entry point. One S3 event may contain several records."""
    results = []
    for record in event.get("Records", []):
        bucket = record["s3"]["bucket"]["name"]
        key = urllib.parse.unquote_plus(record["s3"]["object"]["key"])   # keys arrive URL-encoded
        size = record["s3"]["object"].get("size", 0)
        results.append(process_photo(bucket, key, size))
    return {"results": results}


def process_photo(bucket, key, size):
    """Validates one upload and stores its URL. Returns a short status (also useful in console tests)."""
    if not key.startswith(PREFIX):
        return skip(key, "not under " + PREFIX)
    file_name = key[len(PREFIX):]
    if "/" in file_name or "." not in file_name:
        return skip(key, "expected profile-photos/<username>.<extension>")

    username, extension = file_name.rsplit(".", 1)
    username = username.lower()            # usernames are stored in lower case
    extension = extension.lower()
    if extension not in ALLOWED_EXTENSIONS:
        return skip(key, "." + extension + " is not an accepted image type")
    if size > MAX_SIZE_BYTES:
        return skip(key, "file is larger than 2 MB")

    # Read only the object's metadata (no download). Reached through the S3 gateway endpoint.
    content_type = s3.head_object(Bucket=bucket, Key=key).get("ContentType", "")
    if not content_type.startswith("image/"):
        return skip(key, "Content-Type is " + content_type)

    region = os.environ["AWS_REGION"]      # set by Lambda itself
    photo_url = "https://%s.s3.%s.amazonaws.com/%s" % (bucket, region, urllib.parse.quote(key))

    connection = pymysql.connect(
        host=os.environ["DB_HOST"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        database=os.environ.get("DB_NAME", "user_db"),
        ssl={"ca": CA_BUNDLE},             # encrypted connection, RDS certificate verified
        client_flag=CLIENT.FOUND_ROWS,     # rowcount = matched rows, even when the URL is unchanged
        connect_timeout=5,
    )
    try:
        with connection.cursor() as cursor:
            matched = cursor.execute(
                "UPDATE `user` SET profile = %s WHERE username = %s", (photo_url, username))
        connection.commit()
    finally:
        connection.close()

    if matched == 0:
        return skip(key, "no user with username '" + username + "'")
    logger.info("Profile photo of %s set to %s", username, photo_url)
    return {"key": key, "status": "UPDATED", "username": username, "url": photo_url}


def skip(key, reason):
    """Logs why an upload was ignored. The object stays in S3 and the database is not touched."""
    logger.warning("Skipped %s: %s", key, reason)
    return {"key": key, "status": "SKIPPED", "reason": reason}
```

5.8 .gitignore additions

```
# AWS: private keys, real settings files, Lambda packages
*.pem
*.ppk
app.env
*.zip
```

5.9 Build the JARs and check nothing broke locally

1. In Spring Tools, for each of the five projects: right-click -> **Run As -> Maven build...** -> Goals `clean package` -> Run. Each `target/` folder now contains `<app>-1.0.0.jar` (the `.jar.original` file is not needed).
2. Start everything locally as usual (no profile active). Log in, open a board, create an issue: it must work exactly as before - the `aws` profile is not loaded.
3. Optional local test of the profile: set `SPRING_PROFILES_ACTIVE=aws` with `EUREKA_URL`, `DB_HOST=localhost`, and local credentials in the Run Configuration's Environment tab; the app must start and register its IP in Eureka (visible on http://localhost:8761).

> **CHECKPOINT**
> Five JARs exist; the app still works on the laptop; `git status` shows the new files and no `.pem`, `app.env` or `db.properties`.

PART 6

Milestone 1 - manual deployment, step by step

Rewritten after the real deployment of 19 October 2026 on the company laptop. Every click, command and value in the order that worked, the mistakes that happened and their fixes, what to write down, and what changes when the servers are stopped and started again.

> **ABOUT THE VALUES IN THIS PART**
> This part contains the **addresses** of your deployment (no passwords). Keep the PDF on your laptops; do not upload it to a public place. Every value marked *changes* must be looked up again after a stop/start - section 6.17 explains exactly which ones.

6.0 What is different from the first plan

The first version of this guide was written before anyone touched the console. The table lists everything that turned out differently, so you can explain each decision in the interview.

Area	First plan	What worked (use this)	Why
Laptop	personal laptop paths (`C:\Users\nisha`)	company laptop: project in `...\IC App - AWS\issue-tracking-system`, key in `C:\Users\nlnu19\aws-keys`	only the company laptop has AWS access
SSH rule	SSH 22 from **My IP**	SSH 22 from **Anywhere-IPv4 (0.0.0.0/0)**	the company network sends traffic through a proxy/VPN; the address the console detects is not the address SSH arrives from, so SSH timed out. Login still needs the private key.
RDS template	Free tier or Dev/Test, 20 GiB	**Sandbox** template, db.t3.micro	Dev/Test was refused: "not authorized to perform rds:CreateDBInstance" - the bootcamp policy only allows small instances; Dev/Test also proposed 200 GiB
DB subnet group	two private subnets	exactly **private1 (us-east-1a)** and **private2 (us-east-1b)**	two subnets in the same AZ gave "doesn't meet AZ coverage requirement"
ic-rds-sg	rule from ic-backend-sg	source must be the **security group** `sg-0065c0657e2fd6c17` (ic-backend-sg)	with a wrong source `mysql` hung at the password prompt (packets dropped)
ic-eureka	ic-backend-sg	ic-backend-sg **only**	ic-gateway-public-sg had been selected by mistake; it would publish port 8080 of a private server
Copying files	`scp file host:~`	never forget the `:~` (or `:/path`)	without the colon scp made a **local** copy named like the host
mysql command	console snippet	the guide's command with `--ssl`	the console snippet uses `--ssl-mode`, which the MariaDB client on Amazon Linux does not know
Order	all five servers, then deploy	Eureka deployed alone right after the first SSH test	it needs no database and confirms the whole EC2 setup early
Lambda, CloudWatch, full tests	part of Milestone 1	done **after Milestone 2** (6.14-6.16)	they work for both milestones: the Lambda writes to the shared RDS, CloudWatch covers EC2 and ECS together

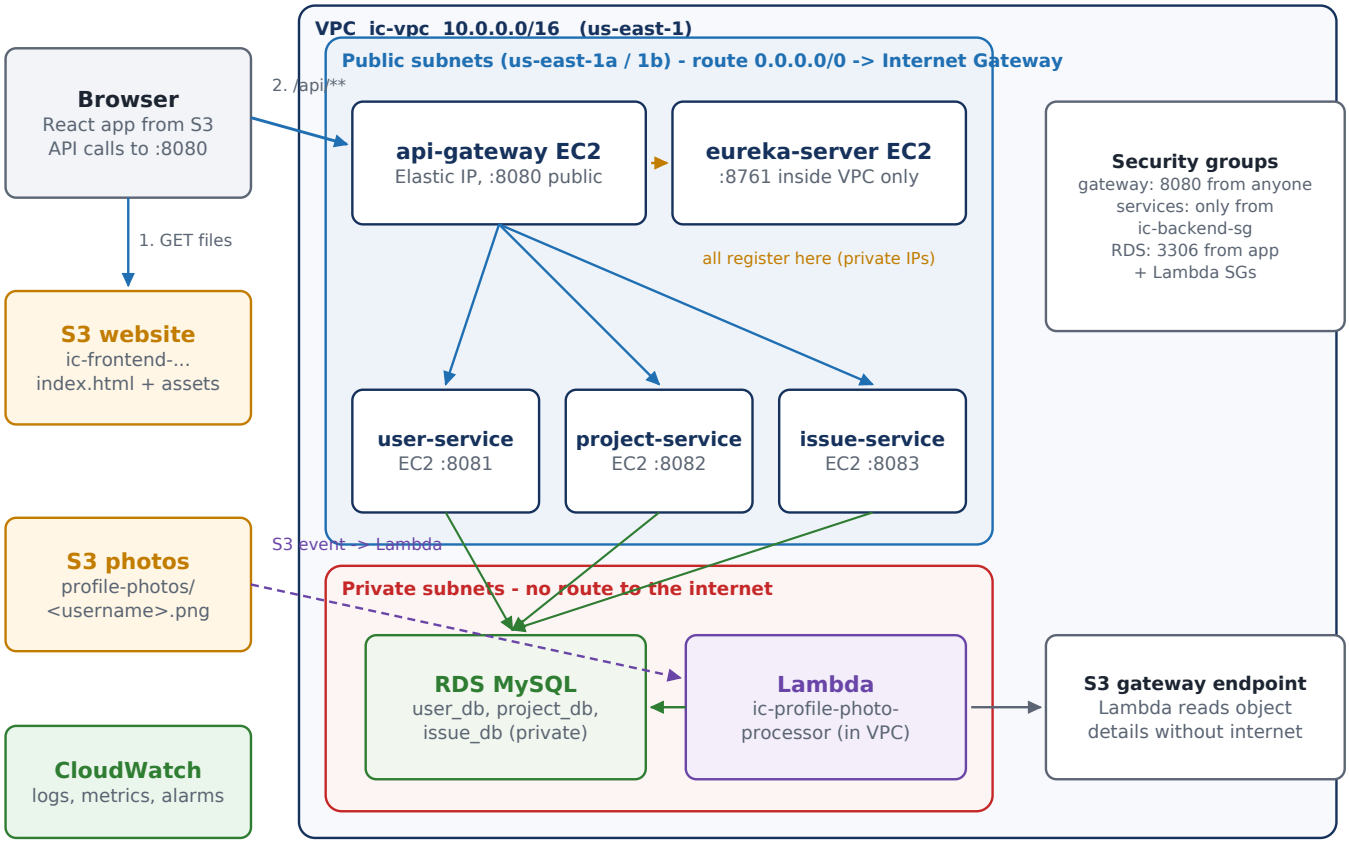


Figure 6.0 - Milestone 1 as deployed: browser -> S3 website; browser -> gateway Elastic IP -> services (found through Eureka) -> private RDS.

6.1 Your notepad worksheet - what to save, what not to save

Create the text file `C:\Users\nlnu19\aws-keys\ic-worksheet.txt` (outside the project, so it can never be committed). Copy table A into it now. Fill each value the moment the console shows it. The values below are those of the deployment of 19 October 2026.

A. Save these - they stay the same until you delete the resource

Value	Your value (19 Oct 2026)	Where to see it again
Region	`us-east-1` (N. Virginia)	top-right of every console page
AWS account ID	`169748358864`	top-right account menu; also inside every ECR image URI
Project folder	`...\IC App - AWS\issue-tracking-system`	File Explorer
Key file	`C:\Users\nlnu19\aws-keys\ic-key.pem`	File Explorer (download happens only once!)
VPC	`ic-vpc` = `vpc-0d7a543bc599a14f4`, 10.0.0.0/16	VPC -> Your VPCs
Subnets	public1 1a 10.0.0.0/20 (`subnet-06d3a5eefc60b1fbc`), public2 1b 10.0.16.0/20, private1 1a 10.0.128.0/20, private2 1b 10.0.144.0/20	VPC -> Subnets
ic-backend-sg	`sg-0065c0657e2fd6c17`	EC2 -> Security Groups
ic-gateway-public-sg	`sg-0decc7a07f80f9702`	EC2 -> Security Groups
Eureka private IP	`10.0.5.157`	EC2 -> ic-eureka -> Private IPv4; also the prompt `ip-10-0-5-157`
Gateway Elastic IP	`52.1.6.53` (private 10.0.27.246)	EC2 -> Elastic IPs
Private IPs of the services	user 10.0.4.115, project 10.0.20.49, issue 10.0.0.66	Eureka dashboard

Value	Your value (19 Oct 2026)	Where to see it again
RDS endpoint	`ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com` (port 3306, user `admin`)	RDS -> ic-mysql -> Connectivity & security -> Endpoints
Frontend bucket	`ic-frontend-nlnu19`	S3 -> Buckets
App address (Milestone 1)	`http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`	bucket -> Properties -> Static website hosting
API address (Milestone 1)	`http://52.1.6.53:8080/api`	built into the React files
Demo logins (not secret)	priya@its.com and karan@its.com / `Owner@123`; rahul@its.com and ananya@its.com / `Assignee@123`	DemoDataLoader.java

B. Do not rely on these - they change on every stop/start

Value	Example (19 Oct 2026)	How to get the current one
Public IP of ic-eureka	13.220.9.243	EC2 -> Instances -> select -> **Public IPv4 address** (copy icon)
Public IP of ic-user	3.239.150.112	same
Public IP of ic-project	18.212.15.96	same
Public IP of ic-issue	3.234.220.128	same
Public DNS names (`ec2-13-220-9-243...`)	follow the public IP	same page, Public IPv4 DNS
Your laptop's internet address	varies (proxy)	not needed - SSH is open to 0.0.0.0/0
ECS task IPs, builder public IP (Milestone 2)	-	ECS -> Tasks; EC2 -> ic-builder

C. Secrets - never in the project, Git, chat or screenshots

Secret	Where it may live
RDS master password (`admin`)	password manager, or your worksheet file in aws-keys
`user_svc`, `project_svc`, `issue_svc` passwords	same; they are also on each server in `/etc/issue-tracker/app.env` (root-only, chmod 600)
`photo_lambda` password	same; later in the Lambda's environment variables
`ic-key.pem`	only `C:\Users\nlnu19\aws-keys` - if lost, you cannot SSH to these servers again
Your copies of the two `.sql` files with real passwords	only aws-keys; delete them on the servers right after running them

> **WATCH OUT**
> Never show `sudo cat /etc/issue-tracker/app.env` during a screen share - it prints the database password. Section 6.19 has a masked version for the demo.

D. No need to save - look it up when needed

Instance IDs, AMI IDs, route-table and endpoint IDs, ARNs of roles. They never appear in a command of this guide.

6.2 Prepare the laptop (once)

1. **Region.** Sign in to the console and choose **United States (N. Virginia) us-east-1** top right. Check it on every page - resources in another region are invisible.
2. **JARs.** In STS, for each of the five projects: right-click -> Run As -> **Maven build...** -> Goals `clean package` -> Run. Each `target` folder then holds `<app>-1.0.0.jar`. Check in PowerShell, in the project folder: `dir .\*\target\*.jar` lists five files.
3. **React dependencies.** VS Code -> open `issue-tracker-ui` -> Terminal -> `npm ci` (once; it must end with 0 vulnerabilities and no error).
4. **Key folder.** Create `C:\Users\nlnu19\aws-keys`. The key and your worksheet live here.
5. **Terminal.** Windows Terminal with PowerShell (or cmd) - both have `ssh` and `scp` built in. Run the `.ps1` helper scripts from **PowerShell** in the project folder. Open a second tab (+) whenever a step says "in another window".

> **CONCEPT**
> Two folders, two jobs: commands that **copy project files** (deploy-jar, build-frontend, copy-to-builder, the Lambda folder) run in the **project folder**; commands that only need the key (ssh, the Eureka tunnel, copying the SQL files) run in **aws-keys**, where `.\ic-key.pem` is enough.

6.3 Key pair

1. EC2 -> **Key pairs** -> Create key pair -> name `ic-key`, type **RSA**, format **.pem** -> Create. The browser downloads `ic-key.pem` once; AWS keeps only the public half.
2. Move it to `C:\Users\nlnu19\aws-keys`.
3. Windows OpenSSH refuses a key that other users can read. In PowerShell, in aws-keys:

```
cd C:\Users\nlnu19\aws-keys
icacls .\ic-key.pem /inheritance:r
icacls .\ic-key.pem /grant:r "$($env:USERNAME):(R)"
```

6.4 Network: the VPC (wizard)

1. VPC console -> **Create VPC** -> **VPC and more**.
2. Name tag auto-generation **on**, value `ic`. IPv4 CIDR `10.0.0.0/16`. No IPv6. Tenancy Default.
3. Number of AZs **2**; public subnets **2**; private subnets **2** (default /20 sizes).
4. NAT gateways **None**. VPC endpoints **S3 Gateway**. DNS hostnames and DNS resolution both **ticked**.
5. **Create VPC** and wait until every step is green.

Subnet (wizard name)	AZ	CIDR	Used by
ic-subnet-public1-us-east-1a	us-east-1a	10.0.0.0/20	ic-eureka, ic-user, ic-issue (M2: ALB, tasks, builder)
ic-subnet-public2-us-east-1b	us-east-1b	10.0.16.0/20	ic-project, ic-gateway (M2: ALB, tasks)
ic-subnet-private1-us-east-1a	us-east-1a	10.0.128.0/20	RDS subnet group, Lambda
ic-subnet-private2-us-east-1b	us-east-1b	10.0.144.0/20	RDS subnet group, Lambda

> **CHECKPOINT**
> Your VPCs -> ic-vpc -> **Resource map**: 4 subnets, the public route table with the Internet Gateway, private route tables with the S3 endpoint.

6.5 IAM role for the EC2 servers

1. IAM -> Roles -> **Create role** -> Trusted entity **AWS service** -> Use case **EC2** -> Next.
2. Tick **CloudWatchAgentServerPolicy** -> Next.
3. Role name `ic-ec2-cloudwatch-role` -> **Create role**.

The role lets the CloudWatch agent on each server send logs and metrics without any access key. If the console ever says "not authorized ... iam:CreateRole", ask the trainer for an existing role.

6.6 Security groups (final, working rules)

EC2 -> Security Groups -> **Create security group**, VPC **ic-vpc** each time. Create them in this order (a rule can only point at a group that exists). To use a group as a source, choose **Custom** in the Source column and type `sg-` or the group name in the search box next to it.

Group	Inbound rules	Attached to
`ic-backend-sg` (sg-0065c0657e2fd6c17)	1) SSH, TCP 22, **Anywhere-IPv4 0.0.0.0/0**. Save. Then **Edit inbound rules** and add: 2) Custom TCP **8761**, source **ic-backend-sg**; 3) Custom TCP **8080-8083**, source **ic-backend-sg**	all five M1 servers
`ic-gateway-public-sg` (sg-0decc7a07f80f9702)	Custom TCP **8080**, Anywhere-IPv4 0.0.0.0/0	**ic-gateway only**
`ic-lambda-sg`	none (outbound only)	the Lambda (later)
`ic-rds-sg`	MySQL/Aurora **3306**, source **ic-backend-sg** (sg-0065c0657e2fd6c17); MySQL/Aurora 3306, source **ic-lambda-sg**	RDS (M2 adds ic-builder-sg and ic-ecs-sg)

WHY THESE RULES

A rule whose source is a **security group** means "any network card that is a member of that group" - not an IP range. When a server is replaced or gets a new address, the rule still works. The rule "8761 from ic-backend-sg" on ic-backend-sg itself means "the members of this group may talk to each other on Eureka's port".

SSH from 0.0.0.0/0 is acceptable here because login needs the private key `ic-key.pem` (Amazon Linux has password login switched off). In a real company you would use Session Manager or a VPN instead of an open port 22.

WATCH OUT

Check the **source column** after saving: it must show the `sg-...` id, not an IP range and not "My IP". The first ic-rds-sg rule had a wrong source and `mysql` hung.

6.7 RDS MySQL

a) DB subnet group

1. RDS -> **Subnet groups** -> Create DB subnet group.
2. Name `ic-db-subnet-group`, description "ic private subnets", VPC **ic-vpc**.
3. Availability Zones: tick **us-east-1a** and **us-east-1b**.
4. Subnets: tick exactly **10.0.128.0/20** (private1, 1a) and **10.0.144.0/20** (private2, 1b) -> Create.

WATCH OUT

"DB subnet group doesn't meet Availability Zone (AZ) coverage requirement" means both chosen subnets are in the same AZ (it happened with private2-1b + public2-1b). RDS needs subnets in at least two AZs even for a Single-AZ database, so it can move it later.

b) Create the database - the settings that the bootcamp account accepts

RDS -> Databases -> **Create database**. Only these fields change; leave the rest as proposed.

Section	Value
Creation method	**Full configuration** (Standard create)
Engine	**MySQL** (MySQL Community); version: the default (8.4.x was used)
Templates	**Sandbox** (on some consoles called Free tier) - **not** Dev/Test or Production
Availability	**Single-AZ DB instance**
DB instance identifier	`ic-mysql`
Master username / credentials	`admin`; **Self managed**; a strong password typed twice (letters, digits and #; avoid `$ \ " ' @ /` and spaces). Write it in your worksheet.
Instance class	Burstable -> **db.t3.micro**
Storage	gp2/gp3, the size the template proposes (20 GiB is plenty); untick **Enable storage autoscaling** if shown
Compute resource	**Don't connect to an EC2 compute resource**
VPC / subnet group	**ic-vpc** / **ic-db-subnet-group**
Public access	**No**
VPC security group	Choose existing -> **ic-rds-sg**; remove `default`
Availability Zone / port	No preference / 3306
Monitoring	Database Insights Standard; Performance Insights off; **Enhanced Monitoring off**
Additional configuration	Initial database name **empty** (the script creates three); backups as the template allows; encryption default; **Deletion protection off** (so you can clean up)

1. **Create database**. Status goes Creating -> Backing-up -> **Available** (about 10-15 minutes). You can launch the EC2 servers (6.8) meanwhile.
2. Open `ic-mysql` -> **Connectivity & security**. In "Connect using" choose **Endpoints** to see the endpoint with a copy button. Copy it into the worksheet:
   `ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com`.

WATCH OUT
"User ... is not authorized to perform: rds:CreateDBInstance ... because no identity-based policy allows" is **not** a network problem. The bootcamp role only allows certain settings. Go back, choose the Sandbox template and db.t3.micro, and create again. If that still fails, look at how batchmates' databases are configured or ask the trainer.

WATCH OUT
Do not use the console's ready-made `mysql ... --ssl-mode=VERIFY_IDENTITY` snippet. Our servers have the **MariaDB** client, which only understands `--ssl`.

CHECKPOINT
Status **Available**; Publicly accessible **No**; VPC security groups **ic-rds-sg (active)**; subnet group with the two private subnets.

6.8 The five EC2 application servers

Launch **ic-eureka first**, test SSH on it, then launch the other four. Each server gets the same first-boot script (`aws\ec2\user-data.sh`, Listing 5.4) with a different line 9.

a) Launch ic-eureka - every field

Section of "Launch an instance"	Value
Name and tags	`ic-eureka`
Application and OS Images	Quick Start -> **Amazon Linux** -> "Amazon Linux 2023 kernel-6.x AMI", **64-bit (x86)** (Free tier eligible)
Instance type	**t3.micro**
Key pair	**ic-key**
Network settings -> **Edit**	VPC **ic-vpc**; Subnet **ic-subnet-public1-us-east-1a**; Auto-assign public IP **Enable**
Firewall (security groups)	**Select existing security group** -> **ic-backend-sg** only (the Summary on the right must say "1 security group")
Configure storage	1 x **8 GiB gp3**; File systems: None
Advanced details -> IAM instance profile	**ic-ec2-cloudwatch-role**
Advanced details -> (everything else)	leave the defaults: Credit specification Unlimited, Metadata version V2 only, hop limit 2
Advanced details -> User data	open `aws\ec2\user-data.sh` in VS Code, change line 9 to `SERVICE=eureka-server`, select all, copy, paste here. Undo the change in VS Code (do not save it).

1. Before clicking **Launch instance**, scroll to the end of the User data box: the last two lines must be `systemctl daemon-reload` and `systemctl enable issue-tracker` (the whole 65-line script was pasted). The `=====` comment lines may look split - that is only the box wrapping long lines.
2. Launch. Wait until **Instance state Running** and **Status check 3/3 checks passed**, then about 3 more minutes for the script (it installs Java, the MySQL client and the CloudWatch agent).

b) First SSH test

```
cd C:\Users\nlnu19\aws-keys
ssh -i .\ic-key.pem ec2-user@<EUREKA-PUBLIC-IP>
#   first time only: "Are you sure you want to continue connecting (yes/no)?" -> type yes
java -version                                 # openjdk "21..." Corretto
systemctl status issue-tracker                # enabled; inactive (dead) = expected, no JAR yet
sudo systemctl status amazon-cloudwatch-agent  # active (running)
sudo tail -n 5 /var/log/cloud-init-output.log  # "Cloud-init ... finished"
exit
```

CONCEPT
The prompt `[ec2-user@ip-10-0-5-157 ~]$` contains the server's **private IP** (10.0.5.157). Write Eureka's into the worksheet - every other service needs it.

WATCH OUT
`Connection timed out` = a firewall dropped the packets. Check: the instance has a public IP; ic-backend-sg has SSH from 0.0.0.0/0; you typed the address without `< >`. `Permission denied (publickey)` = wrong key or user (must be `ec2-user`). `UNPROTECTED PRIVATE KEY FILE` = run the two `icacls` lines of 6.3.

c) Deploy Eureka straight away

Eureka needs no database and its `app.env` stays empty (the first-boot script created the file). In PowerShell, in the **project folder**:

```
cd "C:\Users\nlnu19\Documents\workspace-spring-tools-for-eclipse-5.4.0.RELEASE\IC App - AWS\issue-tracking-system"
dir .\eureka-server\target\eureka-server-1.0.0.jar
powershell -ExecutionPolicy Bypass -File .\aws\scripts\deploy-jar.ps1 -App eureka-server -Server <EUREKA-PUBLIC-IP>
ssh -i C:\Users\nlnu19\aws-keys\ic-key.pem ec2-user@<EUREKA-PUBLIC-IP> "tail -n 20 /var/log/issue-tracker/app.log"
```

CHECKPOINT
The script ends with `Active: active (running)`. The log shows `Started EurekaServerApplication` and then, every 10 seconds, `Running the evict task` - Eureka's normal heartbeat while no service is registered yet.

d) Launch the other four

Exactly as ic-eureka; only these fields change:

Name	Line 9 of user data	Subnet	Security groups
ic-user	`SERVICE=user-service`	public1 (1a)	ic-backend-sg
ic-project	`SERVICE=project-service`	public2 (1b)	ic-backend-sg
ic-issue	`SERVICE=issue-service`	public1 (1a)	ic-backend-sg
ic-gateway	`SERVICE=api-gateway`	public2 (1b)	ic-backend-sg **and** ic-gateway-public-sg

e) Elastic IP for the gateway

1. EC2 -> **Elastic IPs** -> Allocate Elastic IP address -> Allocate.
2. Select it -> Actions -> **Associate Elastic IP address** -> Resource type Instance -> **ic-gateway** -> Associate. Worksheet: gateway Elastic IP **52.1.6.53**.

WHY IT IS DONE THIS WAY
The gateway's address is compiled into the React files. A normal public IP changes on every stop/start; an Elastic IP stays with your account and moves with the instance, so the S3 site keeps working after a restart.

CHECKPOINT
Five instances **Running**, **3/3** status checks; ic-gateway shows the Elastic IP as its public IPv4 address.

6.9 Create the three schemas and the three database users

RDS has no public address, so the SQL runs **from ic-gateway** (a member of ic-backend-sg, which ic-rds-sg accepts). The MySQL client was installed by the first-boot script.

1. Copy the script to aws-keys (PowerShell, in the project folder):

```
copy .\aws\rds\01-schemas-and-users.sql C:\Users\nlnu19\aws-keys\
```

2. Open the **copy** in aws-keys with VS Code or Notepad. Replace the three `CHANGE_ME_...` passwords (letters, digits and `#` only). Save. Write the three passwords in your worksheet - the `app.env` files need them.
3. Copy it to the gateway and run it:

```
cd C:\Users\nlnu19\aws-keys
scp -i .\ic-key.pem .\01-schemas-and-users.sql ec2-user@52.1.6.53:~
ssh -i .\ic-key.pem ec2-user@52.1.6.53
ls                                  # app  01-schemas-and-users.sql
mysql -h ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com -u admin -p --ssl < 01-schemas-and-users.sql
rm 01-schemas-and-users.sql         # it contains passwords
exit
```

WATCH OUT

The `:~` at the end of the scp line is essential. Without it scp copies the file locally to a file named `ec2-user@...` ("1 file(s) copied."). If that happens, delete the stray file (it holds passwords): `del ".\ec2-user@ec2-52-1-6-53.compute-1.amazonaws.com"`.

What you see after typing the admin password	Meaning	Fix
the output below within 1-2 seconds	success	-
`ERROR 1045 ... Access denied for user 'admin'`	wrong master password	type it again; or RDS -> Modify -> New master password -> Apply immediately
nothing happens; after ~2 minutes `ERROR 2002 ... Can't connect`	packets dropped: ic-rds-sg does not allow ic-backend-sg (or RDS is not in ic-vpc)	Ctrl+C; fix the ic-rds-sg source (6.6); Up arrow and run again
`Unknown MySQL server host`	typo in the endpoint	copy it again from the console

```
Database                 user         host
information_schema       issue_svc    10.0.%
issue_db                 project_svc  10.0.%
mysql                    user_svc     10.0.%
performance_schema
project_db
sys
user_db
```

CONCEPT

The `mysql` password prompt appears **before** any connection is made. A wrong password therefore fails instantly, while a network block makes it hang - that is how you tell the two apart. `10.0.%` means the users may connect only from inside ic-vpc.

TRY IT

DBeaver through an SSH tunnel (the class used a public RDS instead): New connection -> MySQL -> Server host = the RDS endpoint, port 3306, user admin. **SSH** tab: Use SSH Tunnel, host 52.1.6.53, user ec2-user, authentication Public key = ic-key.pem. Driver properties: `sslMode=REQUIRED`. DBeaver connects to the gateway by SSH and from there to RDS - the database never needs a public address. Good to show in the interview.

6.10 Deploy user, project, issue and gateway

Order: **user-service -> project-service -> issue-service -> api-gateway** (the demo data of project and issue refer to user ids 1-4). Wait for "Started" in each log before the next.

a) Prepare the four app.env texts in Notepad

Open Notepad (do not save the file in the project). Type the four blocks once; later you copy a block and paste it on the server with a **right-click**.

```
# ---- ic-user
SPRING_PROFILES_ACTIVE=aws
EUREKA_URL=http://10.0.5.157:8761/eureka/
DB_HOST=ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com
DB_USERNAME=user_svc
DB_PASSWORD=<user_svc password>

# ---- ic-project: the same five lines with DB_USERNAME=project_svc and its password
# ---- ic-issue:   the same five lines with DB_USERNAME=issue_svc and its password

# ---- ic-gateway (no DB lines; the localhost origin is replaced in 6.12)
SPRING_PROFILES_ACTIVE=aws
EUREKA_URL=http://10.0.5.157:8761/eureka/
FRONTEND_ORIGIN=http://localhost:5173
```

WHY IT IS DONE THIS WAY

`/etc/issue-tracker/app.env` is read by systemd (`EnvironmentFile=`) and handed to Java as environment variables. The `aws` profile (`application-aws.properties`) turns them into Spring settings: `${EUREKA_URL}`, `${DB_HOST}`... The same JAR therefore runs on the laptop (default profile), on EC2 and in ECS - only the environment differs.

b) For each of the four servers

```
# 1. write the settings on the server   (aws-keys folder)
ssh -i .\ic-key.pem ec2-user@<SERVER-PUBLIC-IP>
sudo nano /etc/issue-tracker/app.env     # right-click = paste;  Ctrl+O, Enter = save;  Ctrl+X = exit
exit

# 2. copy the JAR and (re)start the service   (project folder, PowerShell)
powershell -ExecutionPolicy Bypass -File .\aws\scripts\deploy-jar.ps1 -App user-service -Server <IP>
#    -App: user-service | project-service | issue-service | api-gateway
#    -Server for the gateway: its Elastic IP 52.1.6.53

# 3. after about one minute, read the end of the log
ssh -i C:\Users\nlnu19\aws-keys\ic-key.pem ec2-user@<IP> "tail -n 30 /var/log/issue-tracker/app.log"
```

Log line	Meaning
`Demo users loaded (2 project owners, 2 assignees)`	user-service created its tables in user_db and inserted the demo data (only when the table is empty)
`Registering application PROJECT-SERVICE with eureka with status UP`	the service found Eureka at 10.0.5.157
`registration status: 204`	Eureka accepted the registration
`Started ProjectServiceApplication in 15.893 seconds`	start-up finished
`Demo projects loaded (3 projects)` / `Demo issues loaded (9 issues)`	demo data of the other services
`Discovery Client initialized ... initial instances count: 3`	(gateway) it already sees the three services
`Netty started on port 8080` / `Started ApiGatewayApplication`	the gateway is up
`Resolving eureka endpoints via configuration` (every 5 min)	normal

Error in the log	Cause	Fix
`Access denied for user 'user_svc'@'10.0...'`	password in app.env differs from the SQL copy	`sudo nano` the file again; `sudo systemctl restart issue-tracker`
`Communications link failure`	server cannot reach RDS	server must have ic-backend-sg; ic-rds-sg rule
`Could not resolve placeholder 'DB_HOST'` (or EUREKA_URL...)	line missing or misspelt in app.env	compare with the Notepad block
`Connection refused` to 10.0.5.157:8761 (repeating)	Eureka not running	start ic-eureka / its service first

6.11 Verify the backend

a) The Eureka dashboard through an SSH tunnel

Port 8761 is private. In a **new terminal tab** (aws-keys):

```
ssh -i .\ic-key.pem -L 8761:10.0.5.157:8761 ec2-user@52.1.6.53
```

Leave that window open (closing it, or typing `exit`, closes the tunnel). In the browser open `http://localhost:8761`.

> **CHECKPOINT**
> "Instances currently registered with Eureka": **API-GATEWAY** 10.0.27.246:8080, **ISSUE-SERVICE** 10.0.0.66:8083, **PROJECT-SERVICE** 10.0.20.49:8082, **USER-SERVICE** 10.0.4.115:8081 - all **UP (1)**. The red text "THE SELF PRESERVATION MODE IS TURNED OFF" is expected: we switched it off so stopped services disappear quickly.

> **CONCEPT**
> `-L 8761:10.0.5.157:8761` = "listen on port 8761 of my laptop and forward every connection through the SSH session to 10.0.5.157:8761, as seen from the gateway". The gateway is inside the VPC, so it can reach Eureka's private address.

b) The API through the gateway

Browser: `http://52.1.6.53:8080/api/users` - the four demo users as JSON (`"profile":null` because they have no photo). Optional login test in PowerShell:

```
Invoke-RestMethod -Method Post -ContentType "application/json" `
  -Uri http://52.1.6.53:8080/api/users/login `
  -Body '{"email":"priya@its.com","password":"Owner@123"}'
# answer: userId 1, username priya, role PROJECT_OWNER
```

6.12 The React frontend on S3

a) Build with the gateway address

```
cd "C:\Users\nlnu19\Documents\workspace-spring-tools-for-eclipse-5.4.0.RELEASE\IC App - AWS\issue-tracking-system"
powershell -ExecutionPolicy Bypass -File .\aws\scripts\build-frontend.ps1 -ApiUrl http://52.1.6.53:8080/api
```

It prints `Building the React app with VITE_API_BASE_URL=http://52.1.6.53:8080/api`, then `built in ...s` and the files in `issue-tracker-ui\dist`: `index.html`, `favicon.svg` and the `assets` folder (one .js, one .css, two icon fonts).

CONCEPT
Vite **replaces** `import.meta.env.VITE_API_BASE_URL` with the text `http://52.1.6.53:8080/api` while building. The files on S3 are plain static files - there is no server that could read a setting later. A new API address therefore means a new build and a new upload.

b) The bucket

1. S3 -> **Create bucket** -> General purpose -> name `ic-frontend-nlnu19` (bucket names are global; add a digit if taken) -> region us-east-1.
2. Object Ownership **ACLs disabled**. **Untick "Block all public access"** and tick the acknowledgement. Versioning **Enable**. Encryption SSE-S3 -> **Create bucket**.
3. Open the bucket -> **Upload** -> **Add files**: `index.html` and `favicon.svg` from `dist` -> **Add folder**: `dist\assets` -> **Upload**. `index.html` must be at the top level of the bucket, next to the `assets/` folder - not inside a `dist/` folder.
4. **Permissions** -> Bucket policy -> Edit -> paste Listing 6.1 -> Save.
5. **Properties** -> Static website hosting -> Edit -> **Enable** -> Host a static website -> Index document `index.html` -> Error document `index.html` -> Save. Copy the **Bucket website endpoint**: `http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`.

Listing 6.1 - bucket policy of ic-frontend-nlnu19

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadWebsite",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::ic-frontend-nlnu19/*"
  }]
}
```

WHY IT IS DONE THIS WAY
The error document is `index.html` because React Router handles paths such as `/owner/dashboard` in the browser. S3 has no such file; returning index.html lets the app load and show the right page after F5.

c) Allow the website in the gateway (CORS)

```
ssh -i .\ic-key.pem ec2-user@52.1.6.53
sudo nano /etc/issue-tracker/app.env
#   change only the last line to (no trailing /):
#   FRONTEND_ORIGIN=http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com
sudo systemctl restart issue-tracker
exit
```

CHECKPOINT
Open `http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`: the **InfinityCastle - Issue Tracker** login page. Log in as priya@its.com / Owner@123, role Project Owner: "Good morning, Priya", the board with Customer Portal and the demo issues. F5 on `/owner/dashboard` reloads the page. F12 -> Console: no CORS error.

CONCEPT
The browser shows "Not secure" because S3 website endpoints and port 8080 are plain HTTP. HTTPS needs CloudFront (for S3) and a certificate on a load balancer - see Part 10, "How would you add HTTPS?".

6.13 Milestone 1 status and evidence

Case-study task	Status (19 Oct 2026)	Evidence to keep
React on S3 static website	done	screenshot of the login page and the dashboard with the S3 address bar
One EC2 per microservice (5)	done	EC2 -> Instances, five Running; Eureka dashboard
RDS MySQL, private	done	RDS summary (Available, Publicly accessible No)
Services connected to RDS	done	`/api/users` JSON; logs with "Demo users loaded"
Lambda triggered by S3	after Milestone 2 (6.14)	CloudWatch log line, photo in the app
CloudWatch logs/metrics	agent already sending; dashboard after M2 (6.15)	log groups, dashboard
Testing	basic checks done; full table after M2 (6.16)	table 6.16 with screenshots

Take the screenshots with **Win+Shift+S** (sharper than phone photos) and keep them in a folder outside the project. Crop out anything that shows a password.

6.14 Lambda: the profile-photo processor (do after Milestone 2)

Decided on 19 October: the Lambda, CloudWatch and the full test table are done **after Milestone 2**. The Lambda writes to the shared RDS, so the photo then appears in both the Milestone 1 and the Milestone 2 site. Milestone 1 servers can be stopped meanwhile; ic-gateway must be **running** for steps b) and c).

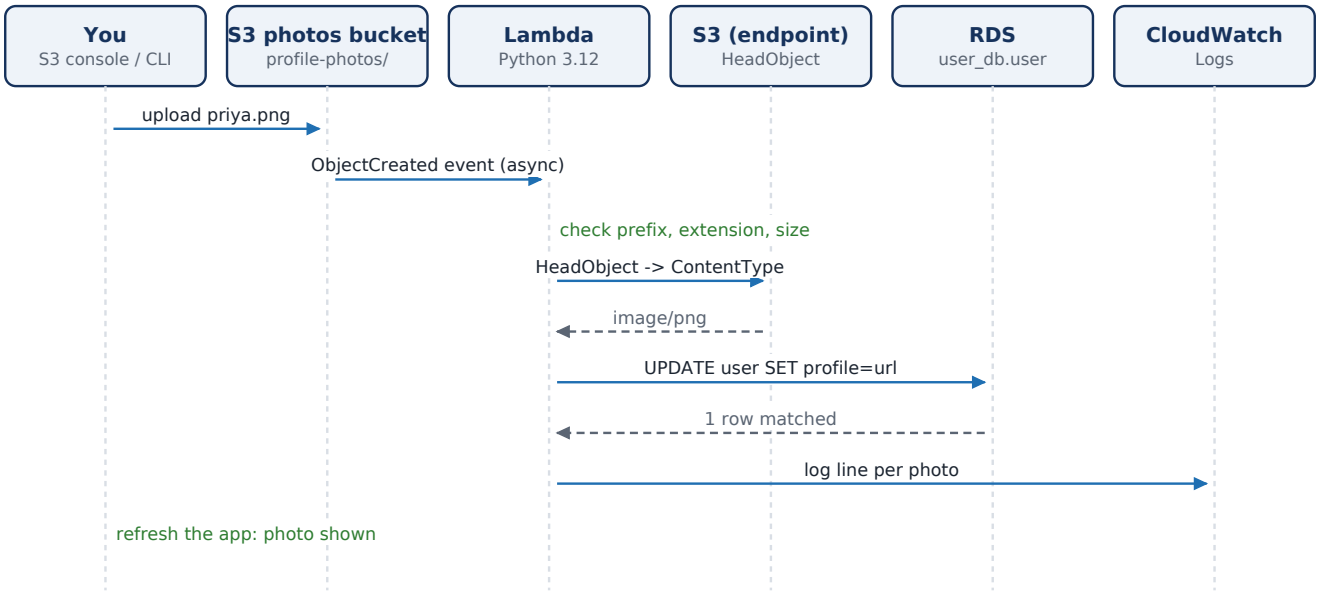


Figure 6.1 - Uploading `profile-photos/priya.png` sets Priya's photo in the app, without any change to the Spring Boot services.

0) One security-group check

ic-rds-sg must have **MySQL 3306 from ic-lambda-sg** (6.6). Add it now if it is missing.

a) The photos bucket

1. S3 -> Create bucket `ic-photos-nlnu19` (us-east-1, ACLs disabled, **untick Block all public access**, versioning on).
2. Permissions -> Bucket policy -> Listing 6.2 (public **read** of `profile-photos/*` only - the browser loads the photos directly from S3).
3. Create folder `profile-photos` (Create folder button).

Listing 6.2 - bucket policy of ic-photos-nlnu19

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadProfilePhotos",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::ic-photos-nlnu19/profile-photos/*"
  }]
}
```

b) Build the deployment package on ic-gateway (it has internet access)

```
# laptop, PowerShell, PROJECT folder: copy the Lambda folder to the gateway
scp -i C:\Users\nlnu19\aws-keys\ic-key.pem -r `
    .\aws\lambda\profile-photo-processor ec2-user@52.1.6.53:~/photo-lambda

# on the gateway: code + PyMySQL + the RDS certificate bundle, all at the root of the zip
ssh -i C:\Users\nlnu19\aws-keys\ic-key.pem ec2-user@52.1.6.53
cd ~/photo-lambda && bash package-lambda.sh
exit

# laptop, aws-keys folder: fetch the zip
cd C:\Users\nlnu19\aws-keys
scp -i .\ic-key.pem ec2-user@52.1.6.53:~/photo-lambda/photo-lambda.zip .
```

WATCH OUT
If the script stops with `$'\r': command not found`, the file got Windows line endings on the way. On the gateway: `sed -i 's/\r$//' package-lambda.sh` and run it again.

c) The database user for the Lambda

```
copy .\aws\rds\02-lambda-user.sql C:\Users\nlnu19\aws-keys\        # project folder
```

1. Set the password in the **copy** in aws-keys; write it in the worksheet.
2. Same three commands as 6.9 with this file name: `scp ... ec2-user@52.1.6.53:~` (with `:~`), `mysql -h ... -u admin -p --ssl < 02-lambda-user.sql`, `rm 02-lambda-user.sql`.
3. The output shows `GRANT SELECT (username), UPDATE (profile) ON user_db.user TO photo_lambda@10.0.%`.

d) The execution role

1. IAM -> Roles -> Create role -> AWS service -> **Lambda** -> attach **AWSLambdaVPCAccessExecutionRole** -> name `ic-photo-lambda-role` -> Create.
2. Open the role -> Add permissions -> **Create inline policy** -> JSON -> Listing 6.3 -> name `ic-read-profile-photos` -> Create.

Listing 6.3 - inline policy of ic-photo-lambda-role

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "ReadProfilePhotoMetadata",
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::ic-photos-nlnu19/profile-photos/*"
  }]
}
```

e) Create the function

1. Lambda -> **Create function** -> Author from scratch -> name `ic-profile-photo-processor` -> Runtime **Python 3.12** -> Architecture x86_64.
2. Permissions -> Change default execution role -> **Use an existing role** -> ic-photo-lambda-role.
3. Additional configurations -> **Enable VPC** -> VPC ic-vpc -> the two **private** subnets -> security group **ic-lambda-sg** -> Create function.
4. Code -> **Upload from -> .zip file** -> `photo-lambda.zip` from aws-keys -> Save. (Handler stays `lambda_function.lambda_handler`.)
5. Configuration -> General configuration -> Edit -> Timeout **15 sec**, memory 128 MB -> Save.
6. Configuration -> Environment variables -> Edit: `DB_HOST` = `ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com`, `DB_NAME` = `user_db`, `DB_USER` = `photo_lambda`, `DB_PASSWORD` = its password -> Save.

f) The S3 trigger

1. S3 -> ic-photos-nlnu19 -> Properties -> **Event notifications** -> Create event notification.
2. Name `ic-photo-uploaded`; Prefix `profile-photos/`; Event types **All object create events**; Destination **Lambda function** -> ic-profile-photo-processor -> Save. (S3 adds the permission to invoke the function.)

g) Test it

1. Rename any small PNG to `priya.png` and upload it into `profile-photos/` of ic-photos-nlnu19.
2. CloudWatch -> Log groups -> `/aws/lambda/ic-profile-photo-processor` -> newest stream: "Profile photo of priya set to https://ic-photos-nlnu19.s3.us-east-1.amazonaws.com/profile-photos/priya.png".
3. In the app, log out and in as Priya (or Ctrl+F5): the sidebar shows the photo instead of "PS".
4. Negative tests: upload `notes.txt` -> "Skipped ... .txt is not an accepted image type"; upload `nobody.png` -> "no user with username 'nobody'". Nothing in the database changes.

Lambda log shows	Cause
time-out on `head_object`	the S3 gateway endpoint is not on the private route tables (VPC -> Endpoints -> Route tables)
time-out on `connect`	ic-rds-sg does not allow ic-lambda-sg (step 0)
`Access denied for user 'photo_lambda'`	password differs between c) and the environment variable
`No module named 'pymysql'`	the zip was built wrongly - repeat b)

6.15 CloudWatch monitoring (do after Milestone 2)

The agent on the five servers has been sending since their first boot, so Milestone 1 logs are already there. One pass covers EC2, ECS, Lambda and RDS together.

1. **Logs:** CloudWatch -> Log groups: `/issue-tracker/m1/<app>` (one per server, made by the agent), `/ecs/ic-<app>` (Milestone 2), `/aws/lambda/ic-profile-photo-processor`. For each: Actions -> **Edit retention setting** -> 14 days (default is "never expire").
2. **Logs Insights:** select the five `/issue-tracker/m1/...` groups (and the `/ecs/...` ones) and run the query below to see every error across the services in one list.
3. **Dashboard** `ic-dashboard`: line widgets for EC2 CPUUtilization (5 instances), IssueTracker/EC2 mem_used_percent, Lambda Invocations + Errors, RDS DatabaseConnections + CPUUtilization, ECS service CPU, ALB RequestCount.
4. **Alarms** (optional, quick): SNS topic `ic-alerts` with your e-mail (confirm the mail). Then `ic-gateway-cpu-high` (CPUUtilization of ic-gateway > 70 for 2 x 5 min), `ic-photo-lambda-errors` (Errors

sum >= 1 in 5 min), `ic-rds-storage-low` (FreeStorageSpace < 2 GB), each notifying ic-alerts.

```
fields @timestamp, @logStream, @message
| filter @message like /ERROR|Exception/
| sort @timestamp desc
| limit 50
```

CHECKPOINT
Each service has a log group with recent lines; the dashboard shows data in every widget; alarms are OK (or "Insufficient data" at first).

6.16 Testing and validation (Milestone 1, task F)

#	Test	Expected
1	Open `http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`	login page; favicon "IC"
2	Sign up a new assignee without a photo, log in	success message; initials in the sidebar
3	Owner: board, search `^add`, filters, project switch, F5	same behaviour as on the laptop
4	Create a project and an issue; edit it	saved; still there after refresh (stored in RDS)
5	Assignee (rahul@its.com / Assignee@123): change a status	owner's board shows the new column
6	Upload `<username>.png` to the photos bucket (after 6.14)	photo appears for that user
7	GET `http://52.1.6.53:8080/api/issues/999`	404 with "Issue not found with id 999"
8	`sudo systemctl stop issue-tracker` on ic-issue, reload the board	friendly "service not available" error + Try again
9	`sudo systemctl start issue-tracker`, wait 30 s, press Try again	board loads (Eureka re-registered it)
10	CloudWatch Logs Insights (after 6.15)	the errors of test 8 are listed
11	EC2 -> ic-user -> Instance state -> **Reboot**	after ~1 min the service is back by itself (systemd, enabled)

6.17 What persists and what changes

The interview may be days after the deployment, and the bootcamp account may stop resources in between. This table tells you, for every event, what survives and what you must redo.

Event	Stays the same	Changes / is lost	What you must do
Reboot an EC2	everything, including the public IP	nothing (the app restarts)	wait 1-2 min; the service is `enabled`, systemd starts it at boot
Stop, then Start an EC2	instance ID, **private IP**, disk (JAR, `app.env`, systemd service, logs, agent config), security groups, role, the **Elastic IP** of ic-gateway	**public IPv4 and public DNS** of ic-eureka, ic-user, ic-project, ic-issue; SSH asks "yes/no" again for the new address	copy the new public IPs into worksheet table B (only needed for SSH and deploy-jar). Nothing in app.env changes - it uses private IPs and the RDS endpoint
Terminate an EC2	nothing of that server	its disk, its private IP; a new server gets new IPs	relaunch it (6.8 a/d), write app.env, deploy the JAR. If it was **ic-eureka**: put its new private IP in EUREKA_URL of all four app.env files and restart them. If it was ic-gateway: associate the Elastic IP again
Stop RDS temporarily	endpoint, schemas, users, data	AWS starts it again by itself after **7 days**	start it before the EC2s and wait for Available (5-10 min)
Delete and recreate RDS with the **same identifier**	endpoint (it is built from identifier + account + region)	all data, users and schemas	run 01 and 02 SQL again; restart the services (they create the tables and demo data)
Elastic IP	the address, while allocated; stays associated over stop/start	released = gone for ever; a new one has another address	never release it before the interview. A new address means a new React build + upload (6.12)

Event	Stays the same	Changes / is lost	What you must do
S3 buckets and website	everything; always on	-	nothing
VPC, subnets, SGs, roles, key pair	everything	-	nothing
Data in RDS	all rows, including what you created during tests	-	demo data is inserted only into empty tables
Eureka registry	-	it is in memory: empty after Eureka restarts	services re-register by themselves within ~30 s
A new day / another network on the laptop	SSH works (0.0.0.0/0)	the SSH tunnel and console login end	open the tunnel again; sign in again
CloudWatch logs	kept (until the retention you set)	-	nothing
ECS service desired tasks 1 -> 0 -> 1 (M2)	task definitions, ALB and its DNS name, Cloud Map name `eureka.ic.local`, ECR images	every task gets a new IP	nothing - tasks re-register in Eureka; `eureka.ic.local` points to the new Eureka task
ic-builder stop/start (M2)	disk with images and files	public IP	use the new IP for SSH and copy-to-builder

WATCH OUT
Ask the trainer whether the bootcamp account **stops or deletes** resources automatically (at night, at the end of a session, or after some days). Stopped is harmless - follow 6.19. Deleted means rebuilding with this part, which is why the worksheet matters.

6.18 Pausing between now and the interview

Option	How	Approximate cost per day	Ready for the demo in
Keep everything running	do nothing	5 x t3.micro ~$1.25 + 5 public IPv4 ~$0.60 + RDS ~$0.41 = ~$2.3	immediately
Stop the five EC2s (recommended)	EC2 -> Instances -> select ic-eureka, ic-user, ic-project, ic-issue, ic-gateway -> Instance state -> **Stop instance**	disks ~$0.10 + Elastic IP ~$0.12 + RDS ~$0.41	~10 minutes (6.19)
Also stop RDS	RDS -> ic-mysql -> Actions -> **Stop temporarily** (if the account allows it)	adds only storage ~$0.08 instead of ~$0.41	~20 minutes; restarts by itself after 7 days

WATCH OUT
Never, before the interview: **terminate** an instance, **release** the Elastic IP, **delete** RDS, the key pair or the buckets. Each of these forces a partial rebuild.

6.19 Demo-day runbook: bring Milestone 1 back and show it

a) Start-up (30 minutes before)

1. Sign in to the console; region **us-east-1**.
2. **RDS first.** RDS -> Databases -> ic-mysql. If the status is *Stopped*: Actions -> **Start** and wait for **Available** (5-10 min). The services cannot start without it.
3. **Eureka next.** EC2 -> Instances -> ic-eureka -> Instance state -> **Start instance**. Wait for 3/3 checks (about 2 min).
4. **The other four together:** select ic-user, ic-project, ic-issue, ic-gateway -> Start instance. Wait for 3/3, then about 2 more minutes - systemd starts every application by itself.

5. **Worksheet table B:** copy the new public IPs of ic-eureka, ic-user, ic-project and ic-issue. ic-gateway keeps 52.1.6.53.
6. **Quick check (browser):** `http://52.1.6.53:8080/api/users` returns the four users. A 503 or "connection refused" in the first minutes means "not registered yet" - wait one more minute.
7. **Eureka tunnel** in a new terminal tab (leave it open), then `http://localhost:8761` in the browser shows the four services UP:

```
cd C:\Users\nlnu19\aws-keys
ssh -i .\ic-key.pem -L 8761:10.0.5.157:8761 ec2-user@52.1.6.53
```

8. **The app:** open `http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com` and log in as Priya. Log out again so the demo starts at the login page.

b) If something is missing

```
ssh -i .\ic-key.pem ec2-user@<PUBLIC-IP-OF-THAT-SERVER>
systemctl status issue-tracker                      # active (running)?
tail -n 40 /var/log/issue-tracker/app.log           # the reason, see the table in 6.10
sudo systemctl restart issue-tracker                # start it again (e.g. after RDS became available)
```

If a service started while RDS was still starting, it failed and systemd retries it every 10 seconds (`Restart=on-failure`) - it usually comes up alone once RDS is Available; a manual restart is quicker.

c) What to show, in this order

#	Show	Say (in your own words)
1	Figure 1.1 (architecture)	S3 serves the React files; the browser calls the gateway on its Elastic IP; the gateway finds the services through Eureka; one private RDS with a schema and a user per service.
2	VPC -> ic-vpc -> Resource map	two public subnets with a route to the Internet Gateway, two private subnets without it, S3 gateway endpoint, two AZs.
3	Security groups: ic-rds-sg, ic-backend-sg	rules reference groups, not IPs; only the gateway's port 8080 is public; RDS accepts only the app servers.
4	RDS -> ic-mysql	Publicly accessible No; endpoint; private subnet group in two AZs; the services connect over TLS (`sslMode=REQUIRED`) with their own limited users.
5	EC2 -> Instances, Elastic IPs	five t3.micro, spread over two AZs; the gateway's fixed address.
6	The app: log in, board, create an issue, F5	data is in RDS; F5 works thanks to the error document.
7	Eureka dashboard via the tunnel	services registered with private IPs (the `aws` profile); Eureka itself is not reachable from the internet.
8	SSH to ic-user: `systemctl cat issue-tracker`, masked app.env, `tail -f` the log while you click in the app	systemd keeps the app running and restarts it; configuration comes from environment variables, one JAR for every environment.
9	Stop issue-service, reload the board, start it, Try again	friendly error, recovery through Eureka re-registration.
10	CloudWatch log groups and dashboard; Lambda photo upload (after 6.14/6.15)	central logs; event-driven serverless step.

Masked view of the settings for step 8 (never `cat` the file on a shared screen):

```
sudo sed 's/PASSWORD=.*/PASSWORD=********/' /etc/issue-tracker/app.env
```

d) After the demo

Pause again (6.18) if more demos may follow, or clean up everything (Part 9.3).

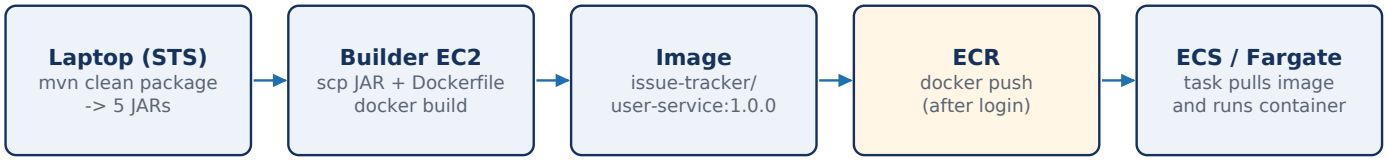
6.20 Problems met during the real deployment

What you saw	Cause	Fix
"DB subnet group doesn't meet Availability Zone (AZ) coverage requirement"	both subnets in us-east-1b	private1 (1a) + private2 (1b)
"not authorized to perform: rds:CreateDBInstance ... no identity-based policy allows"	bootcamp policy rejects the Dev/Test settings (and 200 GiB)	Sandbox template, db.t3.micro
`ssh: connect to host ... port 22: Connection timed out`	SSH rule "My IP" does not match the company network's exit address	SSH 22 from 0.0.0.0/0
`Could not resolve hostname <13.220.9.243>`	the `< >` of the placeholder were typed	type only the address
`scp` printed "1 file(s) copied."	destination without `:~`, so a local copy was made	add `:~`; delete the stray file
`-bash: 01-schemas-and-users.sql: No such file or directory`	consequence of the previous line	copy again with `:~`
`mysql` waits forever after the password	ic-rds-sg did not allow ic-backend-sg	rule MySQL 3306 from sg-0065c0657e2fd6c17
Eureka log: `Running the evict task` every 10 s	normal heartbeat	none
Eureka page: "SELF PRESERVATION MODE IS TURNED OFF"	set on purpose in eureka-server	none
Gateway does not start with the aws profile	`FRONTEND_ORIGIN` not set yet	temporary `http://localhost:5173`, real value in 6.12
Browser: "Not secure"	plain HTTP (S3 website, port 8080)	expected; HTTPS = CloudFront + ACM
ic-eureka had two security groups in the launch summary	ic-gateway-public-sg selected by mistake	remove it before launching

PART 7

Milestone 2 - containers on ECR and ECS Fargate

The same five JARs packaged as Docker images on a builder EC2 (as in class), stored in ECR and run as ECS services on Fargate behind a load balancer.



Same JARs as Milestone 1 - only the packaging and the runtime change

Figure 7.1 - From JAR to running container.

7.1 The builder EC2

1. IAM -> Roles -> Create role -> AWS service -> EC2 -> attach **AmazonEC2ContainerRegistryPowerUser** -> name `ic-builder-role`. (It may push and pull images; it cannot delete repositories.)
2. Security group `ic-builder-sg` (VPC ic-vpc): SSH 22 from **Anywhere-IPv4 0.0.0.0/0** (the company network makes "My IP" unusable - see 6.0). Edit **ic-rds-sg**: add MySQL 3306 from **ic-builder-sg** (for the container test against RDS in 7.4).
3. Launch `ic-builder` like the Milestone 1 servers (6.8 a), except: instance type **t3.medium** (as in class - building images needs memory), subnet public1, security group **ic-builder-sg** only, IAM instance profile **ic-builder-role**, storage **20 GiB** gp3, and this user data:

```
#!/bin/bash
# Builder: Docker for building and testing images
# (the AWS CLI is already part of Amazon Linux 2023)
dnf install -y docker
systemctl enable --now docker
usermod -aG docker ec2-user
```

4. Wait for 3/3 checks plus 2 minutes, then from aws-keys:

```
ssh -i .\ic-key.pem ec2-user@<BUILDER-PUBLIC-IP>
docker --version && aws --version
aws sts get-caller-identity     # account 169748358864 and assumed-role/ic-builder-role - no keys
```

WATCH OUT

`permission denied while trying to connect to the Docker daemon socket`: the `docker` group membership only applies to new logins. Type `exit`, SSH in again, repeat. The builder's public IP changes after every stop/start (worksheet table B).

7.2 Copy the JARs and Dockerfiles (task A.I)

In PowerShell, in the **project folder**, after building the five JARs (6.2), one command copies the five Dockerfiles, the five JARs and `build-and-push.sh`:

```
powershell -ExecutionPolicy Bypass -File .\aws\scripts\copy-to-builder.ps1 -Server <BUILDER-PUBLIC-IP>
```

What it does, step by step (for understanding - you do not type this):

```
$key = "C:\Users\nlnu19\aws-keys\ic-key.pem"
$b    = "<BUILDER-PUBLIC-IP>"
$apps = "eureka-server","api-gateway","user-service","project-service","issue-service"
ssh -i $key ec2-user@$b "mkdir -p ~/build/{eureka-server,api-gateway,user-service}/target ..."
foreach ($a in $apps) {
  scp -i $key ".\$a\Dockerfile"            "ec2-user@${b}:~/build/$a/"
  scp -i $key ".\$a\target\$a-1.0.0.jar"  "ec2-user@${b}:~/build/$a/target/"
}
scp -i $key .\aws\scripts\build-and-push.sh "ec2-user@${b}:~/build-and-push.sh"
```

7.3 Build the images (task A.II)

First create the five ECR repositories (7.5 step 1) - the push needs them. Then, on the builder, `bash build-and-push.sh` (it is in your home folder) builds all five images, logs Docker in to ECR with the instance role, pushes, and prints the five **image URIs** - copy them into the worksheet. To understand or to build only, the build part is:

```
cd ~/build
for app in eureka-server api-gateway user-service project-service issue-service; do
  docker build -t issue-tracker/$app:1.0.0 ./$app
done
docker images            # five issue-tracker/* images, about 300-350 MB each
```

7.4 Test the containers on the builder (task A.III)

All five containers on one Docker network, the services connected to RDS - the same settings the ECS tasks will get. `read -s` keeps passwords out of the shell history.

```
docker network create ic-net
read -p "RDS endpoint: " DB
read -s -p "user_svc pw: " PU; echo
read -s -p "project_svc pw: " PP; echo
read -s -p "issue_svc pw: " PI; echo
COMMON="--network ic-net -e SPRING_PROFILES_ACTIVE=aws -e EUREKA_URL=http://eureka:8761/eureka/ -e DB_HOST=$DB"

docker run -d --name eureka --network ic-net issue-tracker/eureka-server:1.0.0
docker run -d --name user-service $COMMON -e DB_USERNAME=user_svc \
  -e DB_PASSWORD="$PU" issue-tracker/user-service:1.0.0
docker run -d --name project-service $COMMON -e DB_USERNAME=project_svc \
  -e DB_PASSWORD="$PP" issue-tracker/project-service:1.0.0
docker run -d --name issue-service $COMMON -e DB_USERNAME=issue_svc \
  -e DB_PASSWORD="$PI" issue-tracker/issue-service:1.0.0
docker run -d --name api-gateway --network ic-net -p 8080:8080 -e SPRING_PROFILES_ACTIVE=aws \
  -e EUREKA_URL=http://eureka:8761/eureka/ -e FRONTEND_ORIGIN=http://localhost \
  issue-tracker/api-gateway:1.0.0

docker ps                                  # five containers "Up"
docker logs -f user-service                # "Started UserServiceApplication", Ctrl+C
sleep 60; curl -s localhost:8080/api/users | head -c 300; echo
docker rm -f eureka user-service project-service issue-service api-gateway   # clean up
```

Type the four `read` lines one by one: each waits for your input (the endpoint from the worksheet, then the three passwords, which stay invisible). A line ending in `\` continues on the next line - copy both lines together.

CHECKPOINT
The curl through the containerised gateway returns the users from RDS. The images work - anything that fails on ECS later is configuration, not the image.

7.5 ECR: repositories, login, push (task B)

1. ECR -> Private registry -> Repositories -> **Create repository**, five times: `issue-tracker/eureka-server`, `issue-tracker/api-gateway`, `issue-tracker/user-service`, `issue-tracker/project-service`, `issue-tracker/issue-service` (tag mutability **Immutable** is best practice; enable scanning if offered).
2. Login, tag and push - `build-and-push.sh` (7.3) already runs exactly these lines; they are shown so you can explain them or push one image again by hand:

```
REGION=us-east-1
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
REGISTRY=$ACCOUNT.dkr.ecr.$REGION.amazonaws.com
aws ecr get-login-password --region $REGION | docker login --username AWS --password-stdin $REGISTRY

for app in eureka-server api-gateway user-service project-service issue-service; do
  docker tag  issue-tracker/$app:1.0.0 $REGISTRY/issue-tracker/$app:1.0.0
  docker push $REGISTRY/issue-tracker/$app:1.0.0
done
aws ecr list-images --repository-name issue-tracker/user-service --region $REGION
```

CHECKPOINT
Each repository in the console shows the tag `1.0.0`. Copy one **Image URI**: `<account>.dkr.ecr.us-east-1.amazonaws.com/issue-tracker/user-service:1.0.0`.

7.6 Networking for ECS: namespace and security groups

1. **Cloud Map** -> Namespaces -> Create namespace -> name `ic.local` -> Instance discovery **API calls and DNS queries in VPCs** -> VPC ic-vpc -> Create. (Creates a private Route 53 zone visible only inside ic-vpc.)
2. Security group `ic-alb-sg`: HTTP 80 from Anywhere-IPv4.
3. Security group `ic-ecs-sg`: Custom TCP 8080 from **ic-alb-sg**; then edit and add Custom TCP 8761 and 8080-8083 from **ic-ecs-sg** (itself).
4. Edit **ic-rds-sg**: add MySQL 3306 from **ic-ecs-sg**.

7.7 ECS cluster (task C.I)

1. ECS -> Clusters -> **Create cluster** -> name `ic-cluster` -> Infrastructure **AWS Fargate (serverless)** (only).
2. Monitoring: turn on **Container Insights** (small extra CloudWatch cost; gives per-service CPU/memory graphs) -> Create.

7.8 Task definitions (task C.II)

ECS -> Task definitions -> **Create new task definition**, once per application:

Field	Value
Family	`ic-eureka-server`, `ic-user-service`, `ic-project-service`, `ic-issue-service`, `ic-api-gateway`
Launch type	AWS Fargate; OS/Architecture **Linux/X86_64** (images were built on x86)
Task size	CPU **0.5 vCPU**, memory **1 GB**
Task role	none
Task execution role	first time: **Create new role** (ecsTaskExecutionRole); later: choose it
Container - name	the application name, e.g. `user-service`
Container - image URI	`<account>.dkr.ecr.us-east-1.amazonaws.com/issue-tracker/<app>:1.0.0`
Port mappings	container port = the app's port (8761 / 8080 / 8081 / 8082 / 8083), TCP, app protocol HTTP
Environment variables	see the next table
Log collection	**Use log collection -> Amazon CloudWatch** (defaults: group `/ecs/ic-<app>`, created automatically)

Task	Environment variables
ic-eureka-server	(none)
ic-user-service	`SPRING_PROFILES_ACTIVE=aws`, `EUREKA_URL=http://eureka.ic.local:8761/eureka/`, `DB_HOST=<RDS-ENDPOINT>`, `DB_USERNAME=user_svc`, `DB_PASSWORD=...`
ic-project-service	same, `project_svc`
ic-issue-service	same, `issue_svc`
ic-api-gateway	`SPRING_PROFILES_ACTIVE=aws`, `EUREKA_URL=http://eureka.ic.local:8761/eureka/`, `FRONTEND_ORIGIN=http://ic-frontend-m2-nlnu19.s3-website-us-east-1.amazonaws.com` (the Milestone 2 bucket of 7.11)

OPTIONAL HARDENING: SECRETS

Better than a plain-text DB_PASSWORD: store each password in **Secrets Manager** (or SSM Parameter Store SecureString) and in the container's environment choose **ValueFrom** with the secret's ARN. Add a small inline policy to ecsTaskExecutionRole allowing `secretsmanager:GetSecretValue` on those three secrets only. The password then never appears in the task definition.

7.9 The load balancer for the gateway

1. EC2 -> Target groups -> **Create target group** -> target type **IP addresses** -> name `ic-gateway-tg` -> HTTP, port **8080** -> VPC ic-vpc -> Health check path `/actuator/health` -> Next -> do not register targets (ECS will) -> Create.
2. EC2 -> Load balancers -> Create -> **Application Load Balancer** -> name `ic-alb` -> Internet-facing, IPv4 -> VPC ic-vpc -> both AZs with their **public** subnets -> security group **ic-alb-sg** only -> Listener HTTP:80 -> forward to ic-gateway-tg -> Create. Copy the **DNS name**.

7.10 ECS services (tasks C.III and C.IV)

ic-cluster -> Services -> **Create**, in the order eureka -> user -> project -> issue -> gateway. Common settings:

Section	Value
Compute options	**Launch type** FARGATE, platform version LATEST
Deployment configuration	Application type **Service**; family `ic-<app>` (latest revision); service name `ic-<app>-svc`; desired tasks **1**
Networking	VPC ic-vpc; subnets: the two **public** subnets; security group **ic-ecs-sg** only (remove default); **Public IP: turned on** (needed to pull images from ECR without a NAT gateway)
Service discovery - eureka only	Use service discovery -> namespace **ic.local** -> create new service discovery service -> name **eureka** -> DNS record type **A**, TTL 15
Load balancing - gateway only	Application Load Balancer -> **use an existing** ic-alb -> container `api-gateway 8080:8080` -> existing listener 80:HTTP -> existing target group ic-gateway-tg -> **health check grace period 120 s**

CHECKPOINT
All five services show **1/1 tasks running**. Target group ic-gateway-tg shows one **healthy** target (an IP in 10.0.x.x). `http://<ALB-DNS>/api/users` returns the users. The log group `/ecs/ic-user-service` contains "Registering application USER-SERVICE with eureka".

WATCH OUT
Fargate needs 1-2 minutes per Spring Boot task. If the gateway target keeps flipping to unhealthy and the task is replaced, raise the grace period or give the task 1 vCPU. If a service cannot find Eureka, check that the eureka service has the Cloud Map name and that ic-ecs-sg allows 8761 from itself.

7.11 Frontend and RDS for Milestone 2 (task D)

1. RDS: the same instance and schemas as Milestone 1 (task D is already satisfied); the task definitions point at it through `DB_HOST` and the credentials (task D.III).
2. Build with the ALB address, in the project folder (no `:8080` - the ALB listens on port 80):

```
powershell -ExecutionPolicy Bypass -File .\aws\scripts\build-frontend.ps1 -ApiUrl http://<ALB-DNS>/api
```

3. Create a **second** bucket `ic-frontend-m2-nlnu19` exactly as in 6.12 b (its own copy of Listing 6.1 with this name, static website hosting, error document index.html) and upload the new `dist` contents. The Milestone 1 bucket stays untouched, so **both** versions can be demonstrated.
4. The gateway task allows this second website (its `FRONTEND_ORIGIN`, 7.8).
5. Milestone 1 servers: stop them (not terminate) while you do not need them - 6.17 and 6.18.

WHY IT IS DONE THIS WAY
Each React build contains one API address. One bucket per milestone keeps the Milestone 1 site (gateway Elastic IP) and the Milestone 2 site (ALB) working side by side - convenient for the interview.

7.12 CloudWatch, testing and scaling (tasks E and F)

- **Logs:** `/ecs/ic-<app>` groups, one stream per task. Logs Insights works the same as in 6.15.
- **Metrics:** AWS/ECS CPUUtilization and MemoryUtilization per service; Container Insights adds task counts and network; ALB: RequestCount, TargetResponseTime, HTTPCode_Target_5XX_Count, UnHealthyHostCount.
- **Alarms:** `ic-gateway-unhealthy` (UnHealthyHostCount >= 1 for 2 minutes), `ic-user-svc-cpu` (service CPU > 70%), notifying ic-alerts. Add the ECS and ALB graphs to ic-dashboard.
- **End-to-end test:** repeat the Milestone 1 test table through the ALB address.
- **Validate orchestration:** in the ic-user-service-svc service -> Tasks -> stop the running task. ECS starts a replacement within a minute; the new task registers in Eureka; the app keeps working after a short error. That is self-healing.
- **Scale out:** Update service ic-user-service-svc -> desired tasks **2**. Both tasks register in Eureka (different instance ids thanks to the IP in the id) and the gateway spreads calls between them - visible in the two log streams. Optional: Service auto scaling -> min 1, max 3, target tracking ECSServiceAverageCPUUtilization 60%.

```
# Optional: look at the registry from the builder (add 8761 from ic-builder-sg to ic-ecs-sg first)
curl -s -H "Accept: application/json" http://eureka.ic.local:8761/eureka/apps/USER-SERVICE | head -c 600
```

PART 8

Milestone 3 - GitHub

Commit the deployment changes and push the build to the repository - Deloitte GitHub Enterprise, as the case-study guidelines require.

8.1 What goes into Git

Commit	Never commit
All source code, `application-aws.properties`, Dockerfiles, `aws/` scripts with `CHANGE_ME` placeholders, `app.env.example`, `.env.production` (contains only the public API address), README updates	`*.pem`, `*.ppk`, real `app.env`, `db.properties`, SQL files with real passwords, `photo-lambda.zip`, `target/`, `node_modules/`, `dist/`, AWS access keys

8.2 Steps

```
cd issue-tracking-system
git status                          # check: no .pem, app.env, db.properties, zip in the list
git switch -c feature/aws-deployment
git add .
git commit -m "AWS deployment: aws profile, configurable CORS, Dockerfiles, Lambda, scripts"
git push -u origin feature/aws-deployment
# on GitHub Enterprise: open a Pull Request into main, review the diff, merge
git switch main && git pull
git tag v1.0-aws && git push origin v1.0-aws
```

1. If the repository does not exist yet: create it in the Deloitte GitHub Enterprise organisation (no README, no .gitignore - the project has them), then `git remote add origin <https URL>` and push `main` first.
2. Add a "Deployment" section to the main README: the architecture figures, the environment variables of each application, and a link to this runbook.

WATCH OUT
If a secret was ever committed, deleting the file later does not remove it from the history. Change the password (RDS: Modify -> new master password; MySQL: `ALTER USER ... IDENTIFIED BY ...`) and rotate any key.

PART 9

Troubleshooting, cost and clean-up

The problems you are most likely to meet, what causes them and how to fix them; what costs money; and how to remove everything safely.

9.1 Troubleshooting

Symptom	Likely cause	Fix
`ssh` times out	SSH rule limited to "My IP" (company proxy); instance in a private subnet; no public IP	SSH 22 from 0.0.0.0/0 in ic-backend-sg / ic-builder-sg; public subnet with auto-assign public IP
"UNPROTECTED PRIVATE KEY FILE"	Windows file permissions on ic-key.pem	the two `icacls` commands in 6.3
`mysql` hangs after the password	ic-rds-sg does not allow the caller's security group	add MySQL 3306 from that group (6.6)
scp says "1 file(s) copied."	destination without `:~` - local copy	add `:~`; delete the stray file
Service exits at start: "Could not resolve placeholder EUREKA_URL"	app.env missing a line or the profile variable	check `/etc/issue-tracker/app.env`; `sudo systemctl restart issue-tracker`
"Communications link failure" to MySQL	ic-rds-sg does not allow the caller's SG; wrong endpoint	add the SG rule; copy the endpoint again
"Access denied for user 'user_svc'@'10.0...'"	wrong password, or the user was created for another host	run the SQL again / `ALTER USER`
Eureka lists services as `localhost`	`aws` profile not active	`SPRING_PROFILES_ACTIVE=aws` in app.env or the task definition
Gateway returns 503 for /api/...	target service not registered yet or stopped	wait 30 s; check Eureka and the service log
Browser: "blocked by CORS policy"	`FRONTEND_ORIGIN` differs from the address bar (http vs https, trailing slash, other bucket endpoint form)	copy the website endpoint exactly; restart the gateway / new task revision
React app shows "Cannot reach the server"	wrong `VITE_API_BASE_URL` in the build, gateway down, port 8080 not open	rebuild with the right address; check ic-gateway-public-sg
404 when refreshing a page on S3	error document not `index.html`	Static website hosting -> Error document index.html
403 on the S3 website	Block Public Access still on or no bucket policy	turn off BPA for the bucket; add Listing 6.1
Lambda time-out	S3 endpoint missing on private route tables, or RDS SG	see the table in 6.14 g
Lambda "No module named pymysql"	library not at the root of the zip	rebuild: `pip3 install --target .` inside the folder
ECS task stops: "CannotPullContainerError"	no public IP on the task, wrong image URI, or missing execution role	turn Public IP on; copy the URI from ECR; select ecsTaskExecutionRole
ECS gateway task keeps restarting	ALB health check fails during start-up	grace period 120-180 s; health path /actuator/health
Out of memory / very slow on t3.micro	1 GiB is tight for the JVM + OS	`-Xmx320m`, or t3.small
"not authorized to perform iam:CreateRole"	bootcamp account restrictions	ask the trainer for an existing role

9.2 What costs money (approximate, us-east-1)

Resource	Approximate price	Tip
EC2 t3.micro x5 / t3.medium builder	~$0.0104 / ~$0.0416 per hour each	stop them when not demonstrating
Public IPv4 addresses (incl. Elastic IP)	~$0.005 per hour each	release the Elastic IP after the demo
RDS db.t3.micro + 20 GB	~$0.017 per hour + storage	stop it (up to 7 days) or delete with a snapshot
Fargate 0.5 vCPU / 1 GB	~$0.025 per hour per task	set desired tasks to 0 between demos
Application Load Balancer	~$0.0225 per hour + usage	delete after Milestone 2
S3, Lambda, ECR, CloudWatch	cents at this scale	set log retention (14 days)
NAT gateway (not used)	~$0.045 per hour + data	the design avoids it

Prices change and differ by region; check the AWS pricing pages and the Billing console. Free-tier and bootcamp credits may cover part of it.

9.3 Clean-up order

1. ECS: set every service to 0 tasks, delete the five services, then the cluster; deregister task definitions (optional).
2. Delete the ALB, then the target group. Delete the Cloud Map namespace (after the eureka service is gone).
3. ECR: delete the five repositories (with their images).
4. Lambda: delete the function; S3: remove the event notification; empty and delete both buckets.
5. EC2: terminate the six instances; **release the Elastic IP**; delete the key pair if no longer needed.
6. RDS: take a final snapshot, then delete `ic-mysql` (and the subnet group).
7. Delete the security groups, the IAM roles created here, the SNS topic, alarms, dashboard and log groups.
8. Delete ic-vpc last (this removes subnets, route tables, the Internet Gateway and the S3 endpoint).

PART 10

Interview questions and answers

Questions about AWS, the design and the case study, with short answers to give in your own words.

Architecture and design

Q. Walk me through your Milestone 1 architecture.

React on an S3 static website; five Spring Boot apps on five EC2s in a VPC - Eureka, the gateway with an Elastic IP, and three services; one private RDS MySQL with a schema and a user per service; a Lambda triggered by S3 photo uploads that writes the photo URL to RDS; CloudWatch agent logs and metrics, dashboards and alarms.

Q. What changes between Milestone 1 and 2?

Packaging and runtime: the same JARs become Docker images in ECR, run as ECS services on Fargate. An ALB replaces the Elastic IP, Cloud Map gives Eureka a DNS name, logs go to CloudWatch through the awslogs driver, and ECS restarts and scales tasks.

Q. Why did the code need changes for AWS?

Every service registered itself in Eureka as "localhost" and used a local database. The `aws` profile reads the Eureka URL and RDS settings from environment variables and registers the private IP. The gateway reads its CORS origin from configuration. One JAR then runs everywhere.

Q. Why one RDS instance for three services?

Cost and simplicity. Isolation is kept with three schemas and three MySQL users, each limited to its own schema.

Q. Why Eureka and not Cloud Map for everything?

No code change, and the same model in both milestones. Cloud Map only gives Eureka a stable address. Replacing Eureka with Cloud Map or ECS Service Connect is a valid AWS-native alternative.

Q. Why Fargate instead of the EC2 launch type?

No servers to patch or scale; pay per task; the case study asks for it. The EC2 launch type is cheaper at large, steady scale.

Networking and security

Q. What makes a subnet public?

Its route table has `0.0.0.0/0` pointing to an Internet Gateway. Resources also need a public IP.

Q. Security group vs network ACL?

A security group is stateful, allow-only, on a network interface, and can reference other groups. A NACL is stateless, allows and denies, numbered rules, on a subnet.

Q. How did you apply least privilege?

Security groups that reference each other (RDS only from the app, Lambda and ECS groups); only the gateway/ALB public; roles instead of keys; a Lambda role that reads one prefix of one bucket; per-service DB users; a column-level grant for the Lambda.

Q. How does the Lambda reach S3 without internet?

Through an S3 gateway VPC endpoint - a route in the private route tables. It is free, unlike a NAT gateway.

Q. How would you add HTTPS?

A domain in Route 53, a certificate from ACM, an HTTPS listener on the ALB, and CloudFront in front of the S3 bucket (with Origin Access Control, so the bucket can be private).

Q. Is your API secure?

Internal services are not reachable from the internet, and DB connections are encrypted. Per-request authentication (JWT or Cognito) is the planned next step - the API does not check tokens yet.

Services

Q. What is an IAM role and why use it on EC2?

An identity with temporary credentials that a service assumes. The CloudWatch agent and Docker push work without storing access keys on the server.

Q. What happens when S3 invokes your Lambda twice for the same upload?

Nothing harmful. The function is idempotent: it sets the same URL again (FOUND_ROWS makes it report a match).

Q. What is a cold start?

The first invocation creates the execution environment and loads the code; later calls reuse it. We create the S3 client outside the handler for that reason.

Q. Task definition vs task vs service?

Blueprint vs running copy vs the controller that keeps N copies running, replaces failed ones and connects them to the load balancer.

Q. Task role vs task execution role?

The execution role is used by ECS to pull images and write logs; the task role is for the application's own AWS calls (none needed here).

Q. Multi-AZ vs read replica?

Multi-AZ is a synchronous standby for failover (availability). A read replica is an asynchronous copy for read scaling.

Q. How are backups handled?

RDS automated backups with 7-day point-in-time restore, plus manual snapshots before changes; S3 versioning; images versioned in ECR; code in Git.

Q. Scalability vs elasticity in your system?

Scalability: we can add tasks (user-service to 2). Elasticity: ECS auto scaling adds and removes tasks automatically on CPU.

Q. How do you monitor it?

CloudWatch Logs per service (agent on EC2, awslogs on ECS, built in for Lambda); metrics for EC2/RDS/Lambda/ECS/ALB; a dashboard; alarms to an SNS e-mail topic; Logs Insights queries for errors.

APPENDIX A

Remaining files, copy-paste ready

The files referred to in Part 5 that are not printed there in full.

A.1 - project-service/src/main/resources/application-aws.properties

```
# ================================================================================================
# AWS profile of project-service - loaded in addition to application.properties when SPRING_PROFILES_ACTIVE=aws.
# Every value that differs between the laptop and AWS comes from an environment variable, so the SAME
# JAR runs locally (default profile), on EC2 (Milestone 1) and on ECS Fargate (Milestone 2).
# ================================================================================================

# Eureka registry address, e.g. http://10.0.3.25:8761/eureka/ (EC2) or http://eureka.ic.local:8761/eureka/ (ECS)
eureka.client.service-url.defaultZone=${EUREKA_URL}
# Register this instance by its private IP instead of "localhost", so other machines can call it
eureka.instance.prefer-ip-address=true
# Unique per running copy - two tasks of the same service must not share an instance id
eureka.instance.instance-id=${spring.application.name}:${spring.cloud.client.ip-address}:${server.port}

# Amazon RDS (MySQL). DB_HOST = RDS endpoint. The schema is created by 01-schemas-and-users.sql.
# sslMode=REQUIRED: the connection to MySQL is encrypted.
spring.datasource.url=jdbc:mysql://${DB_HOST}:3306/project_db?sslMode=REQUIRED
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

A.2 - issue-service/src/main/resources/application-aws.properties

```
# ================================================================================================
# AWS profile of issue-service - loaded in addition to application.properties when SPRING_PROFILES_ACTIVE=aws.
# Every value that differs between the laptop and AWS comes from an environment variable, so the SAME
# JAR runs locally (default profile), on EC2 (Milestone 1) and on ECS Fargate (Milestone 2).
# ================================================================================================

# Eureka registry address, e.g. http://10.0.3.25:8761/eureka/ (EC2) or http://eureka.ic.local:8761/eureka/ (ECS)
eureka.client.service-url.defaultZone=${EUREKA_URL}
# Register this instance by its private IP instead of "localhost", so other machines can call it
eureka.instance.prefer-ip-address=true
# Unique per running copy - two tasks of the same service must not share an instance id
eureka.instance.instance-id=${spring.application.name}:${spring.cloud.client.ip-address}:${server.port}

# Amazon RDS (MySQL). DB_HOST = RDS endpoint. The schema is created by 01-schemas-and-users.sql.
# sslMode=REQUIRED: the connection to MySQL is encrypted.
spring.datasource.url=jdbc:mysql://${DB_HOST}:3306/issue_db?sslMode=REQUIRED
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

A.3 - eureka-server/Dockerfile

```
# Container image of eureka-server (Milestone 2).
# Build context = the eureka-server folder, after "mvn clean package" created target/eureka-server-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/eureka-server-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8761

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

A.4 - api-gateway/Dockerfile

```
# Container image of api-gateway (Milestone 2).
# Build context = the api-gateway folder, after "mvn clean package" created target/api-gateway-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/api-gateway-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8080

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

A.5 - project-service/Dockerfile

```
# Container image of project-service (Milestone 2).
# Build context = the project-service folder, after "mvn clean package" created
» target/project-service-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/project-service-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8082

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

A.6 - issue-service/Dockerfile

```
# Container image of issue-service (Milestone 2).
# Build context = the issue-service folder, after "mvn clean package" created target/issue-service-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/issue-service-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8083

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

A.7 - aws/lambda/profile-photo-processor/requirements.txt

```
pymysql>=1.1,<2
```

APPENDIX B

Command cheat sheet

Task	Command
SSH	`ssh -i ic-key.pem ec2-user@<public-ip>`
SSH via the gateway to a private address	`ssh -i ic-key.pem -J ec2-user@<gateway-eip> ec2-user@<private-ip>`
Tunnel a private port to the laptop	`ssh -i ic-key.pem -L 8761:<eureka-private-ip>:8761 ec2-user@<gateway-eip>`
Copy a file up / down	`scp -i ic-key.pem file ec2-user@<ip>:~` / `scp -i ic-key.pem ec2-user@<ip>:~/file .`
Service control	`sudo systemctl start|stop|restart|status issue-tracker`
Follow the app log	`tail -f /var/log/issue-tracker/app.log`
First-boot output	`sudo tail -n 50 /var/log/cloud-init-output.log`
CloudWatch agent status	`sudo systemctl status amazon-cloudwatch-agent`
MySQL client to RDS	`mysql -h <rds-endpoint> -u admin -p --ssl`
Who am I in AWS	`aws sts get-caller-identity`
Docker basics	`docker images`, `docker ps -a`, `docker logs -f <name>`, `docker rm -f <name>`
Build an image	`docker build -t issue-tracker/<app>:1.0.0 ./<app>`
ECR login	`aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin <account>.dkr.ecr.us-east-1.amazonaws.com`
Tag + push	`docker tag issue-tracker/<app>:1.0.0 <registry>/issue-tracker/<app>:1.0.0` then `docker push ...`
List images in ECR	`aws ecr list-images --repository-name issue-tracker/<app> --region us-east-1`
React production build	`npm run build` (reads `.env.production`)